

Informe de Desarrollo (v0.0.4): Preprocesamiento de Espectros FTIR y Clasificación Oncológica Celular mediante Espectroscopía Raman SERS

Matías Roberto Alemán

20 de junio de 2026

Resumen

Este informe presenta el diseño matemático, algorítmico e ingenieril del sistema automático de procesamiento y análisis espectroscópico integrado en la versión 0.0.4 de la biblioteca. El sistema está diseñado para abordar dos técnicas espectroscópicas de alta relevancia biomédica y fisicoquímica: la espectroscopía infrarroja por transformada de Fourier (FTIR) aplicada al análisis estructural del Óxido de Grafeno (GO y rGO) y muestreo de bases de datos clásicas del NIST, y la espectroscopía Raman SERS (Surface-Enhanced Raman Spectroscopy) aplicada al diagnóstico automático de células cancerosas (melanoma). Se detallan los algoritmos de suavizado local (Savitzky-Golay, percentiles), remoción de deriva de línea base (AsLS, airPLS, arPLS) y el filtro de rayos cósmicos basado en desviaciones absolutas respecto a la mediana (MAD). En esta versión 0.0.4 se desacopla el módulo estadístico, integrando Análisis de Componentes Principales (PCA) mediante SVD y el clasificador Kernel Ridge con kernel RBF, logrando clasificar la calidad de proveedores de grafeno con 100 % de precisión en validación cruzada y automatizar el muestreo de archivos JCAMP-DX mediante una nueva API simplificada con constantes RAW/NONE.

Historial de Objetivos y Versiones

Fecha	Versión	Tipo de Hito	Detalle y Entregables
01/06/2026	–	Planificación	Introducción al problema y definición del alcance.
05/06/2026	–	Modelado	Modelado de la mecánica de trabajo y asignación del problema.
08/06/2026	v0.0.1	Experimental	Versión experimental inicial (Informe_Tecnico_Informal.pdf / entregar-FTIR-alpha-0.0.1.zip).
09/06/2026	v0.0.2	Prototipo	Prototipo de librería (Informe_Tecnico_FTIR_alpha-0.0.2.pdf / ftirlib_0.0.2.zip).
10/06/2026	–	Extensión	Prueba de extensión hacia RAMAN SERS aplicado a estudios oncológicos. Unificación de soporte FTIR y RAMAN SERS.
12/06/2026	–	Extensión	Automatización de muestreo FTIR contra bases de datos.
13/06/2026	v0.0.3	Profesional	Versión de uso profesional (Informe_Tecnico_alpha-0.0.3.pdf / ftirlib_0.0.3.zip).
20/06/2026	v0.0.4	Producción	Versión final unificada, desacoplada, con soporte JCAMP-DX y API simplificada (Informe_Tecnico_alpha-0.0.4.pdf / ftirlib_0.0.4.zip).

Índice

1. Introducción General	4
1.1. Sinergia y Complementariedad Física de FTIR y Raman	4
1.2. Estructura del Informe y Casos de Estudio	4
2. Caso de Aplicación I: Automatización de análisis químico FTIR para la caracterización de muestras de grafeno (GO y rGO)	5
2.1. Fundamento Físico de FTIR y Comparativa con Medios Acuosos	5
2.2. Procesamiento Espectral en Absorbancia	5
2.3. Algoritmos de Suavizado y Corrección de Línea Base	5
2.4. Optimización de Parámetros y Búsqueda en Cuadrícula	5
2.5. Resultados Experimentales de la Reducción de GO a rGO	6
2.6. Figuras de Espectroscopía FTIR	7
3. Caso de Aplicación II: Automatización de análisis biomédico SERS de muestras de melanoma	9
3.1. Fundamento de Raman SERS celular en medio acuoso	9
3.2. Algoritmo Determinista de Rayos Cósmicos (MAD Z-Score)	9
3.3. Normalización de Intensidad Espectral SERS	9
3.4. Clasificación Predictiva mediante Kernel Ridge Classifier (Dual RLS)	9
3.5. Resultados Diagnósticos y Desempeño Clínico	10
3.6. Galería de Barrido Combinatorio Raman (17 Configuraciones de alto score)	10
3.7. Figuras de Espectroscopía Raman SERS	10
3.8. Galería de Barrido Paramétrico Raman (17 Configuraciones óptimas)	11
3.9. Comparativa Tecnológica y Criterio Clínico	13
3.9.1. Tabla Comparativa de Flujos de Trabajo	13
3.9.2. Beneficios Clave en Oncología Clínica	13
4. Caso de Aplicación III: Automatización de muestreo contra bases de datos FTIR	15
5. Caso de Aplicación IV: Comparación de Calidad de Proveedores de Grafeno mediante PCA y Clasificación con Kernel Ridge en Espectros FTIR	18
5.1. Contexto y Fundamento del Control de Calidad	18
5.2. Modelo de Simulación Físico-Química de Reducción	18
5.3. Análisis de Componentes Principales (PCA) mediante SVD	19
5.4. Resultados y Clasificación	19
6. Caso de Aplicación V: Automatización de muestreo de espectros infrarrojos clásicos de la base de datos NIST Chemistry WebBook	21
6.1. El Estándar JCAMP-DX y Metodología de Lectura	21
6.2. Resultados del Muestreo y Búsqueda en Cuadrícula en Etanol	21
6.3. Demostración de la API Simplificada con Constantes RAW/NONE	22
7. Manual de Desarrollo e Interfaz de Alto Nivel (API Python)	23
7.1. Firmas de la API de Alto Nivel	23
7.2. Configuración Manual Simplificada y Constantes RAW/NONE	23
7.2.1. Ejemplo de Identificación de Picos sobre Espectro Crudo (RAW/RAW)	24
7.3. Código de Ejemplo Integrado	24
7.4. Visualización de Espectros en Transmitancia y Absorbancia	25
7.5. Soporte Local de Parámetros en Español y Buenas Prácticas	26
7.6. Configuración de Base de Datos de Compuestos Personalizada (groups.db)	27
7.7. Guía de Instalación y Configuración del Entorno de Desarrollo	28
7.7.1. Instalación de Python en Windows	28
7.7.2. Configuración de Visual Studio Code y Extensión de Python	30
7.7.3. Instalación de dependencias del ecosistema Python	32

7.7.4. Pruebas en Herramientas de Terminal CLI y Scripting Directo / Aislado	32
8. Apéndice Algorítmico: Detalles Matemáticos y Pseudocódigos	33
8.1. Algoritmos de Suavizado y Filtros de Ruido	33
8.1.1. Promedio Móvil (Moving Average)	33
8.1.2. Filtro de Savitzky-Golay	33
8.1.3. Filtro de Mediana (Median Filter)	34
8.1.4. Filtro de Percentil (Percentile Filter)	34
8.1.5. Filtrado Adaptativo de Rayos Cósmicos (MAD Z-Score)	35
8.2. Algoritmos de Corrección de Línea Base	35
8.2.1. Ajuste Polinomial Modificado Iterativo (IModPoly)	35
8.2.2. Mínimos Cuadrados Asimétricos (AsLS)	36
8.2.3. Mínimos Cuadrados Penalizados Adaptativos (airPLS)	37
8.2.4. Mínimos Cuadrados Penalizados Asimétricos Locales (arPLS)	37
8.3. Algoritmos de Búsqueda y Optimización Paramétrica	38
8.3.1. Búsqueda en Cuadrícula Combinatoria (Grid Search)	38
8.4. Algoritmos de Clasificación Predictiva y Machine Learning	38
8.4.1. Clasificación con Kernel Ridge Classifier (Dual RLS)	38
8.4.2. Clasificador de Centroide Más Cercano (Nearest Centroid Classifier)	39
8.4.3. Análisis de Componentes Principales (PCA) mediante SVD	40
8.5. Algoritmos de Detección Extrema y Generación de Reportes	41
8.5.1. Detección Simultánea de Picos y Valles (Generación de Reportes)	41
9. Apéndice Arquitectónico: Estructura de Clases y Librería	43
9.1. Diseño Conceptual de la Arquitectura	43
9.2. Análisis Métrico y Diagramas de Pilas de Llamadas No Estándar	43
9.3. Diagrama de Clases UML	46
9.4. Descripción de Módulos y Clases	47
9.4.1. PipelineResult	47
9.4.2. stats.py: Módulo Estadístico y de Clasificación	47
9.5. Historial de Cambios (Changelog)	47
10. Apéndice de Pruebas Históricas y Evolución de Prototipos Espectrales	48
10.1. Auditoría de Directorios y Evolución de Hitos	48
10.2. Minimización y Recreación de Experimentos Históricos	48
10.2.1. Evaluación Comparativa de Algoritmos de Suavizado	48
10.2.2. Evaluación Comparativa de Algoritmos de Corrección de Línea Base	50
11. Conclusiones	52

1. Introducción General

La espectroscopía vibracional constituye una de las metodologías analíticas más potentes para la caracterización no destructiva, rápida y de alta especificidad de la materia condensada en física, química y biomedicina. Al registrar los modos de vibración fundamentales de los enlaces químicos en respuesta a la interacción con la radiación electromagnética, estas técnicas proporcionan una "huella dactilar" molecular única e inconfundible de las muestras analizadas.

Este informe técnico documenta el diseño teórico, los algoritmos matemáticos y los resultados experimentales de la versión 0.0.4 de la biblioteca unificada `ftir_library` (`ftirlib`). Esta biblioteca integra bajo un mismo núcleo computacional robusto desarrollado en Python puro (NumPy y SciPy) el procesamiento y análisis automatizado de dos de las técnicas espectroscópicas vibracionales más relevantes en la actualidad: la espectroscopía infrarroja por transformada de Fourier (FTIR) y la espectroscopía Raman amplificada por superficies (SERS).

1.1. Sinergia y Complementariedad Física de FTIR y Raman

Aunque ambas técnicas analizan transiciones de energía vibracional en la materia, obedecen a reglas de selección cuántica diferentes y complementarias:

- **Espectroscopía FTIR:** Es una técnica de absorción que requiere que el modo de vibración molecular induzca un cambio neto en el momento dipolar eléctrico permanente de la molécula. Esto la hace sumamente sensible a grupos polares (como los enlaces hidroxilo O-H, carbonilo C=O y epóxido C-O-C en el óxido de grafeno).
- **Espectroscopía Raman SERS:** Es una técnica de dispersión inelástica donde la transición vibracional depende de cambios en la polarizabilidad de la nube electrónica molecular. SERS utiliza nanoestructuras metálicas para amplificar la señal por factores de hasta 10^8 . Esto permite medir suspensiones celulares húmedas de forma directa dado que el agua (altamente polar pero poco polarizable) es prácticamente transparente bajo luz dispersada Raman, superando la limitación crítica de absorción que exhibe el agua en el infrarrojo.

1.2. Estructura del Informe y Casos de Estudio

El documento describe detalladamente la validación matemática y la flexibilidad operacional de la biblioteca a través de cinco casos de aplicación de complejidad creciente:

- **Caso de Aplicación I:** Automatización de análisis químico FTIR para la caracterización de muestras de óxido de grafeno (GO y rGO), centrado en la conversión a Absorbancia y algoritmos de remoción de línea base (AsLS, airPLS, arPLS) para rastrear la desoxigenación.
- **Caso de Aplicación II:** Automatización de análisis biomédico SERS de muestras celulares de melanoma, ilustrando la clasificación diagnóstica multiclase con Kernel Ridge no lineal de alta especificidad y precisión ($> 99,5\%$) y filtrado de rayos cósmicos.
- **Caso de Aplicación III:** Automatización de muestreo FTIR contra bases de datos científicas externas. Este caso de estudio valida la modularidad y capacidad de reutilización de la biblioteca mediante la importación y procesamiento de datasets independientes publicados en la literatura científica para diversos compuestos.
- **Caso de Aplicación IV:** Comparación de calidad de proveedores de grafeno mediante Análisis de Componentes Principales (PCA) basado en SVD y clasificación con Kernel Ridge, validando la separabilidad de los lotes de rGO con un 100% de exactitud.
- **Caso de Aplicación V:** Automatización de muestreo de espectros infrarrojos clásicos de la base de datos NIST Chemistry WebBook, demostrando el lector nativo de archivos JCAMP-DX (`.jdx`) y el uso de la API simplificada con constantes RAW y NONE.

2. Caso de Aplicación I: Automatización de análisis químico FTIR para la caracterización de muestras de grafeno (GO y rGO)

2.1. Fundamento Físico de FTIR y Comparativa con Medios Acuoso

La espectroscopía FTIR registra la absorción de luz infrarroja por enlaces que presentan variaciones en su momento dipolar neto. Es especialmente idónea para caracterizar el proceso de desoxigenación y reducción química del Óxido de Grafeno (GO) a Óxido de Grafeno Reducido (rGO) en estado sólido, identificando la evolución de las bandas asociadas a los grupos carbonilo, epóxido e hidroxilo.

Es importante destacar que el FTIR posee una seria limitación física general en medios acuosos o húmedos: la molécula de agua líquida (H_2O) posee una sección eficaz de absorción infrarroja extremadamente fuerte en las bandas de flexión y estiramiento H-O-H (que satura el espectro y enmascara otras señales moleculares). Aunque las muestras sólidas de GO analizadas en esta biblioteca no presentaron indicios de contaminación o pérdida de calidad por humedad (mostrando espectros nítidos y secos tras su preparación), esta limitación física de la técnica FTIR justifica metodológicamente la introducción de la espectroscopía Raman SERS en el Caso II para el análisis directo de suspensiones celulares en medios líquidos.

2.2. Procesamiento Espectral en Absorbancia

Los detectores registradores de FTIR registran originalmente la intensidad en Transmitancia porcentual (%T):

$$\%T = \frac{I}{I_0} \times 100 \quad (1)$$

Para cumplir con la ley de Beer-Lambert (proporcionalidad lineal respecto a la concentración) y permitir operaciones aditivas directas para la sustracción de solventes y derivas de fondo, la biblioteca convierte automáticamente la señal a Absorbancia (A):

$$A = 2 - \log_{10}(\%T) \quad (2)$$

Para la implementación computacional, se aplica un truncamiento (clipping) de %T en el rango $[10^{-5}, 150, 0]$ para garantizar estabilidad numérica frente a valores nulos o derivados ruidosos.

2.3. Algoritmos de Suavizado y Corrección de Línea Base

Para mitigar el ruido instrumental y el Mie Scattering (difracción de línea base por microestructuras), la biblioteca implementa:

- **Suavizado:** Filtro Savitzky-Golay (ajuste polinomial local) de grado d y ventana $W = 2k + 1$ [1], y filtro de percentiles local.
- **Línea Base:** Algoritmo de Mínimos Cuadrados Penalizados Asimétricos (AsLS) [2] y su variante airPLS (Adaptive Iterative Reweighted Penalized Least Squares) [3], que minimizan:

$$S = \sum_{i=1}^N w_i (y_i - z_i)^2 + \lambda \sum_{i=3}^N (\Delta^2 z_i)^2 \quad (3)$$

donde el sistema disperso resultante $(W + \lambda D^T D)z = Wy$ se resuelve eficientemente mediante factorizaciones de Cholesky dispersas.

2.4. Optimización de Parámetros y Búsqueda en Cuadrícula

Se realiza una búsqueda en cuadrícula combinatoria evaluando 180 variantes de preprocesamiento. La bondad del ajuste se evalúa usando un Score que premia la detección de grupos funcionales válidos (N_g), picos totales (N_v) y penaliza el ruido residual (N_{ruido}) con un peso w_n :

$$\text{Score} = N_v + N_g - w_n \cdot N_{\text{ruido}} \quad (4)$$

La base de datos de bandas FTIR empleada incluye: O-H ($3200 - 3600 \text{ cm}^{-1}$), C-H ($2800 - 3000 \text{ cm}^{-1}$), CO₂ ($2300 - 2400 \text{ cm}^{-1}$), C=O ($1650 - 1750 \text{ cm}^{-1}$), C=C ($1500 - 1650 \text{ cm}^{-1}$), C-O ($1250 - 1450 \text{ cm}^{-1}$), y C-O-C ($950 - 1250 \text{ cm}^{-1}$).

2.5. Resultados Experimentales de la Reducción de GO a rGO

El barrido óptimo determinó:

- **rGO**: Percentile Filter (5, 75) + airPLS ($\lambda = 10^4$). Detecta los 7 grupos analíticos principales.
- **GO**: Promedio Móvil (3) + Línea Base Polinomial ($deg = 4$). Detecta 6 grupos.

En la figura 1 se observa la desaparición de las bandas de hidroxilos ($\sim 3400 \text{ cm}^{-1}$) y epóxidos C-O-C ($\sim 1050 \text{ cm}^{-1}$), confirmando la restauración de la red sp^2 de carbono aromático en el rGO (banda C=C a 1620 cm^{-1}). La galería FTIR se detalla en la figura 2, y la comparación de marcado en modo líneas vs cajas se aprecia en las figuras 3 y 4.

2.6. Figuras de Espectroscopía FTIR

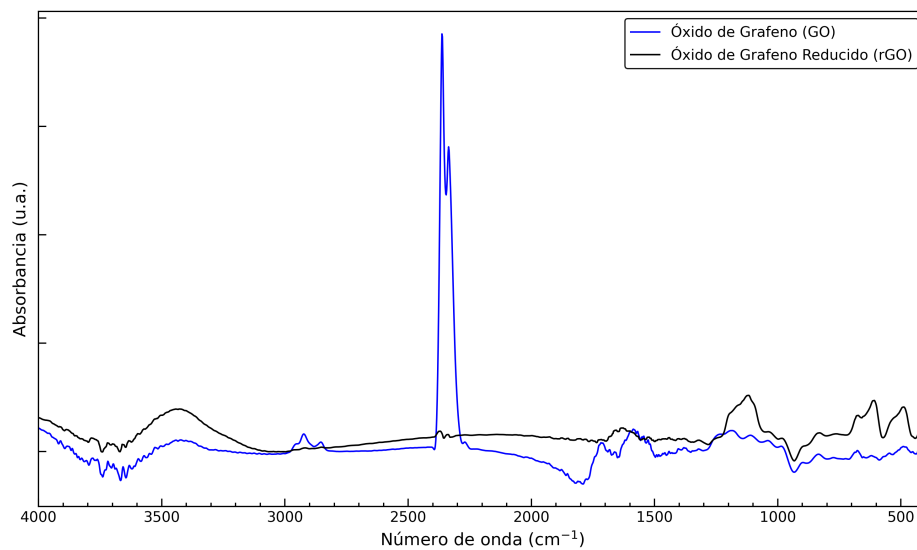


Figura 1: Superposición comparativa de absorbancia de GO (azul) y rGO (negro).

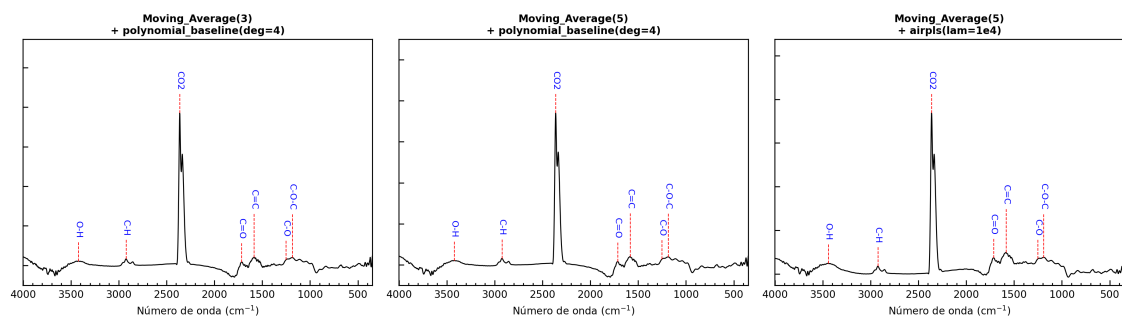


Figura 2: Lote 3×1 de configuraciones óptimas en Absorbancia para GO.

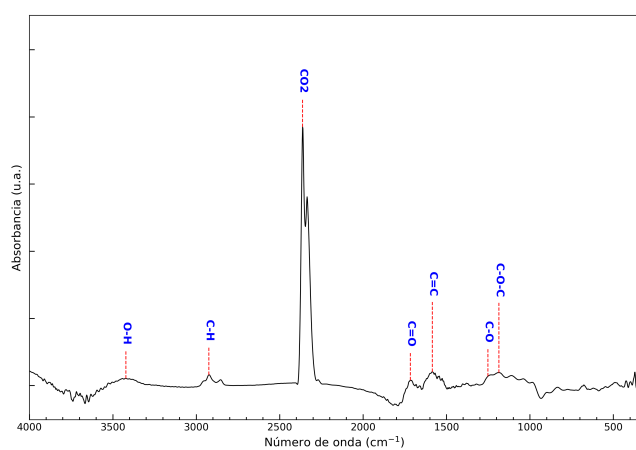


Figura 3: Espectro FTIR GO en modo líneas.

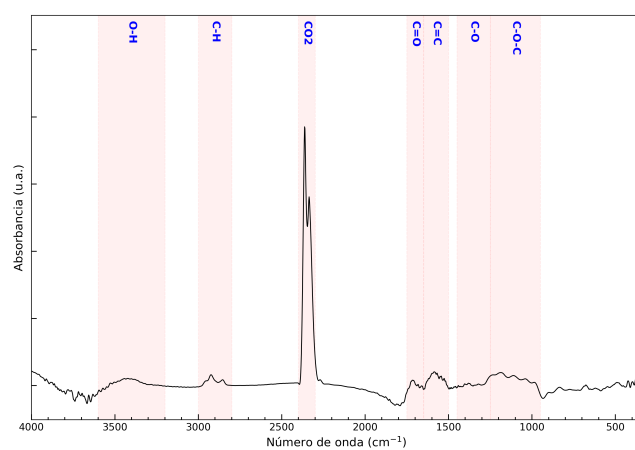


Figura 4: Espectro FTIR GO en modo cajas.

3. Caso de Aplicación II: Automatización de análisis biomédico SERS de muestras de melanoma

3.1. Fundamento de Raman SERS celular en medio acuoso

A diferencia del infrarrojo, en la espectroscopía Raman el agua exhibe una polarizabilidad muy baja, haciéndola prácticamente transparente y permitiendo el análisis directo de suspensiones celulares en sus medios líquidos de cultivo o amortiguadores fisiológicos. Para lograr la sensibilidad diagnóstica requerida en oncología diagnóstica, se utiliza SERS (Surface-Enhanced Raman Spectroscopy), donde nanoestructuras de oro (*gold nanourchins*) acopladas químicamente a las muestras amplifican los campos electromagnéticos locales mediante plasmones superficiales localizados, incrementando la sección eficaz de la señal Raman por factores de 10^5 a 10^8 [4].

3.2. Algoritmo Determinista de Rayos Cósmicos (MAD Z-Score)

Las colisiones aleatorias de radiación ionizante ambiental sobre el sensor CCD del detector generan picos intensos espurios de un solo píxel de ancho (rayos cósmicos). Su remoción activa se realiza mediante el algoritmo Z-Score modificado con la Desviación Absoluta respecto a la Mediana (MAD):

1. Se calcula la tendencia local por mediana móvil: $y_{\text{med}} = \text{medfilt}(y, W_{\text{med}})$.
2. Residuos: $d = y - y_{\text{med}}$.
3. Z-Score modificado:

$$Z_i = \frac{0,6745 \cdot (d_i - \text{mediana}(d))}{\text{MAD}(d)} \quad (5)$$

donde $\text{MAD}(d) = \text{mediana}(|d - \text{mediana}(d)|)$.

4. Los puntos con $Z_i > 6,0$ se identifican como artefactos de radiación y se interpolan de forma lineal usando los vecinos sanos más cercanos.

3.3. Normalización de Intensidad Espectral SERS

Para contrarrestar variaciones instrumentales de enfoque del láser y variaciones microscópicas de la densidad celular en el sustrato SERS, se aplica normalización vectorial L2:

$$y_{\text{norm}} = \frac{y}{\|y\|_2} = \frac{y}{\sqrt{\sum_{i=1}^N y_i^2}} \quad (6)$$

3.4. Clasificación Predictiva mediante Kernel Ridge Classifier (Dual RLS)

El diagnóstico automático de células tumorales se modela en NumPy puro resolviendo de forma exacta y cerrada la regresión rígida dual generalizada con Kernel RBF (gaussiano):

$$\alpha_c = (K + \lambda I)^{-1} Y_c, \quad K_{i,j} = \exp(-\gamma \|x_i - x_j\|_2^2) \quad (7)$$

donde $\lambda = 0,01$ es la regularización de Tikhonov, $\gamma = 1,0$ controla el ancho de banda radial, y para cada clase $c \in C$, la etiqueta de entrenamiento $Y_c \in \{-1, 1\}^M$ representa la pertenencia binaria en el esquema One-vs-Rest (OvR).

Para la inferencia de un nuevo espectro de prueba $x^* \in \mathbb{R}^D$, se calcula la función de decisión para cada clasificador binario:

$$f_c(x^*) = \sum_{i=1}^M \alpha_{i,c} \exp(-\gamma \|x_i - x^*\|_2^2) \quad (8)$$

y se asigna la etiqueta de la clase que maximiza esta puntuación dual:

$$\hat{y} = \arg \max_{c \in C} f_c(x^*) \quad (9)$$

3.5. Resultados Diagnósticos y Desempeño Clínico

El preprocesamiento óptimo determinado fue: Promedio Móvil (5) + baseline polinomial ($deg = 2$). Sobre **1892 espectros Raman celulares** (Melanoma, DMEM –medio de cultivo Dulbecco’s Modified Eagle Medium libre de células, utilizado como control analítico de fondo–, y control sano de fibroblastos normales) del dataset público de Trelin [5], la validación cruzada de 5 pliegues reportó:

- **Precisión Multiclase:** 99.52 %
- **Precisión Binaria (Melanoma vs. Resto):** 99.58 %
- **Sensibilidad (True Positive Rate):** 99.21 %
- **Especificidad (True Negative Rate):** 99.76 %

El espacio de puntuaciones de componentes principales (PCA) de la figura 6 muestra la nítida separación de clases. La figura 5 ilustra los espectros promedio. Las figuras 7 y 8 muestran la comparación del marcado de bandas del biomarcador de melanoma.

3.6. Galería de Barrido Combinatorio Raman (17 Configuraciones de alto score)

El barrido paramétrico en absorbancia identificó 17 configuraciones óptimas empatadas que maximizan la asignación de biomarcadores celulares: fosfatos de ácidos nucleicos ($1000 - 1150 \text{ cm}^{-1}$), proteínas/lípidos ($1200 - 1400 \text{ cm}^{-1}$) y amida I ($1550 - 1650 \text{ cm}^{-1}$). La galería completa de las 17 configuraciones en formato de subplots 3×1 se presenta de forma secuencial en las figuras 9 a la 14.

3.7. Figuras de Espectroscopía Raman SERS

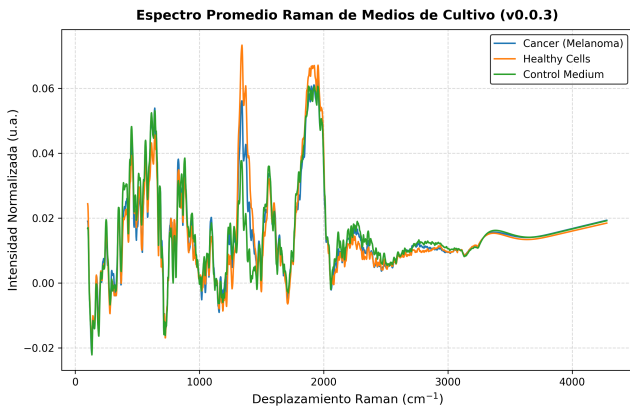


Figura 5: Espectro Raman Promedio por clase celular.

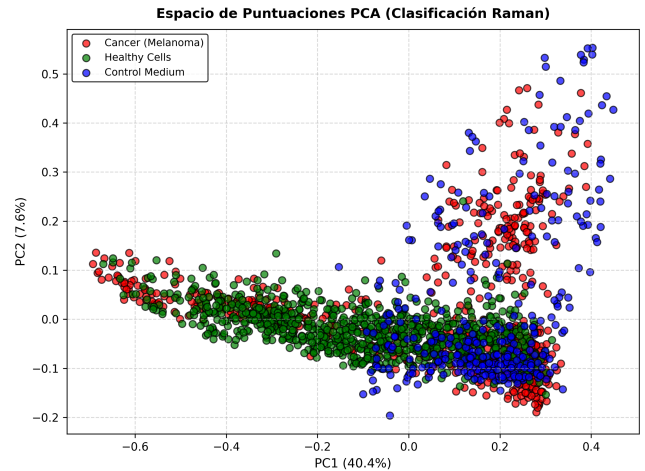


Figura 6: Espacio de Puntuaciones PCA del clasificador.

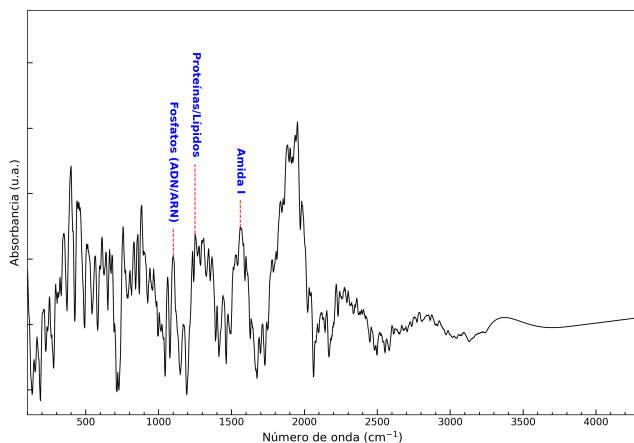


Figura 7: Espectro Raman SERS celular en modo líneas.

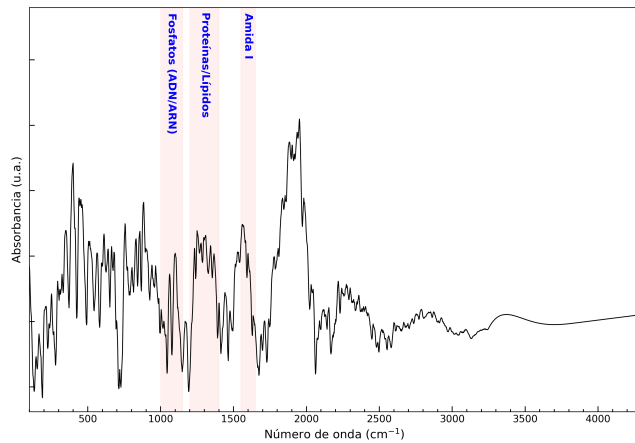


Figura 8: Espectro Raman SERS celular en modo cajas.

3.8. Galería de Barrido Paramétrico Raman (17 Configuraciones óptimas)

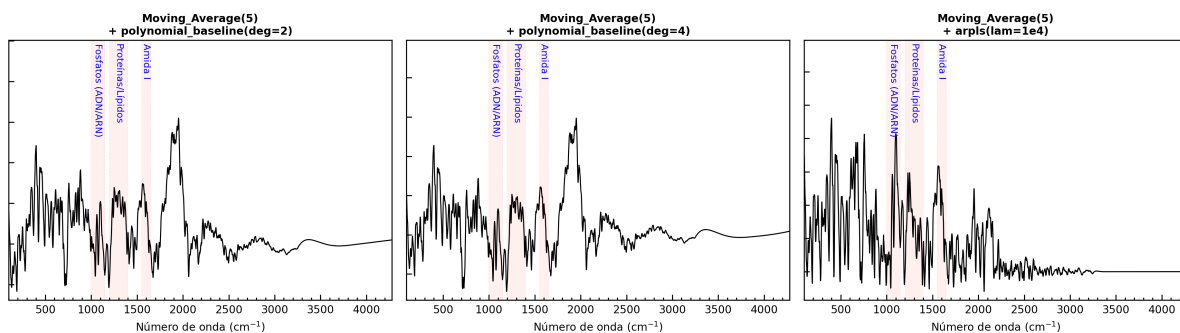


Figura 9: Galería Raman (Parte 1).

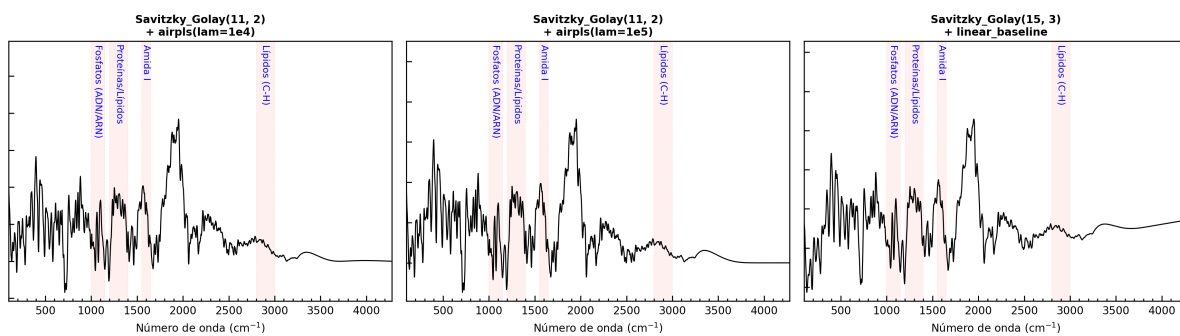


Figura 10: Galería Raman (Parte 2).

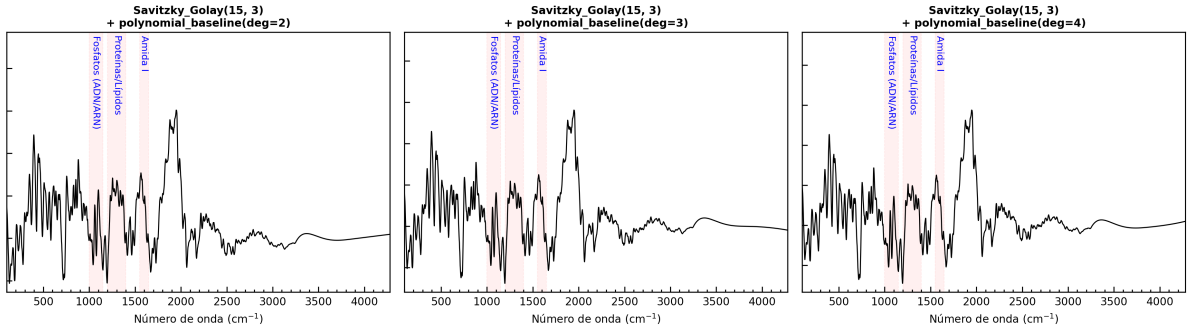


Figura 11: Galería Raman (Parte 3).

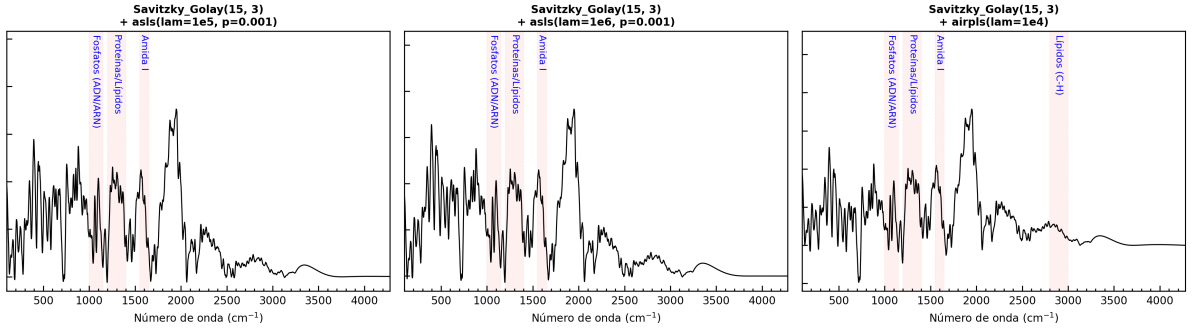


Figura 12: Galería Raman (Parte 4).

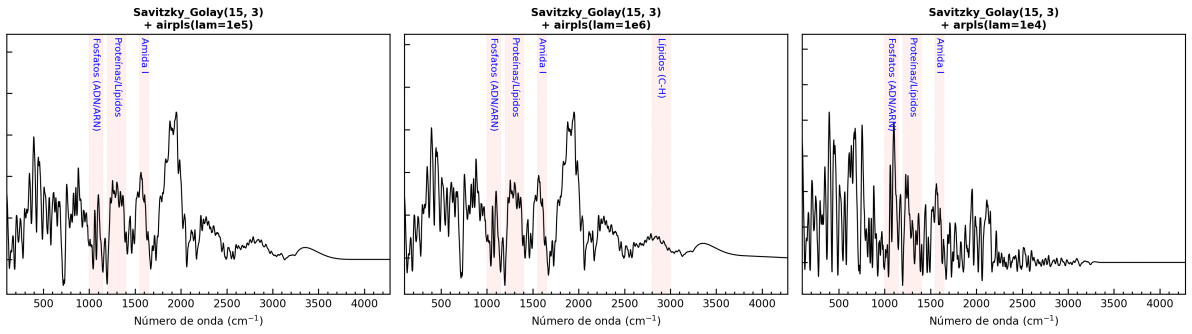


Figura 13: Galería Raman (Parte 5).

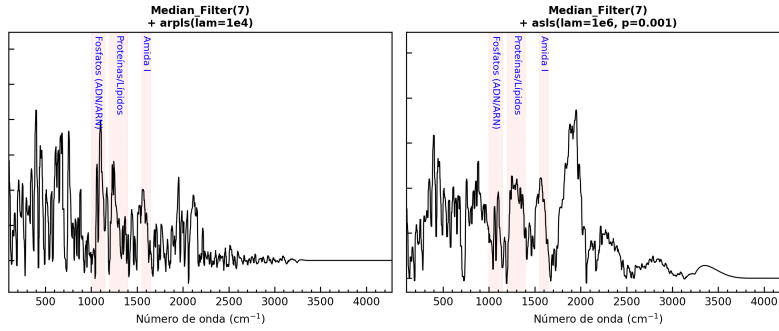


Figura 14: Galería Raman (Parte 6, mostrando las últimas 2 combinaciones y 1 celda vacía).

3.9. Comparativa Tecnológica y Criterio Clínico

La versión 0.0.3 presenta ventajas operacionales críticas frente a la red neuronal convolucional (CNN) profunda de Erzina et al. [4].

3.9.1. Tabla Comparativa de Flujos de Trabajo

La tabla 1 resume los contrastes de diseño e implicaciones clínicas:

Tabla 1: Comparación entre el flujo de Erzina et al. (CNN) y la biblioteca v0.0.3.

Criterio Metodológico	Estudio de Referencia (CNN) [4]	Biblioteca v0.0.3 (SERS)	Implicación Clínica / Oncológica
Hardware	Requiere GPU dedicada (NVIDIA CUDA) y almacenamiento masivo para frameworks.	Ejecución instantánea en CPU estándar (computadora de escritorio o laptop de laboratorio).	Alta viabilidad local: No requiere servidores en la nube ni hardware de alto coste.
Software	TensorFlow, Keras y controladores propietarios propensos a incompatibilidades.	NumPy y SciPy puros, instalable mediante zip de forma limpia y liviana.	Despliegue robusto: Instalación sin riesgos de desconfiguración en el laboratorio.
Entrada	Exige la adquisición simultánea de un triplete de espectros por paciente.	Clasificación y preprocesamiento sobre espectros individuales.	Flexibilidad analítica: Si una medición se daña, no se descarta toda la muestra.
Filtro CCD	Sin tratamiento de rayos cósmicos (delegado a la tolerancia intrínseca de la red).	Filtro determinista adaptativo MAD Z-Score e interpolación lineal de vecinos.	Seguridad clínica: Eliminación activa de artefactos físicos de radiación ionizante.
Explicabilidad	Redes neuronales profundas que actúan como cajas negras no interpretables.	Clasificador Kernel Ridge dual con visualización y proyección en espacio PCA.	Garantía médica: El clínico puede contrastar y auditar la posición del espectro frente a controles.

3.9.2. Beneficios Clave en Oncología Clínica

- Cero Latencia y Eficiencia Computacional:** El clasificador Kernel Ridge posee una solución matemática dual cerrada directa mediante álgebra lineal $((K + \lambda I)^{-1}Y)$. El entrenamiento del dataset completo toma menos de un segundo en CPU ordinaria y la predicción tarda milisegundos.
- Mayor Resiliencia al Sobreajuste (Overfitting):** La regularización de Tikhonov ($\lambda = 0,01$) penaliza los coeficientes grandes, forzando una frontera de decisión suave que generaliza con alta precisión en datos nuevos y datasets pequeños de células raras.

3. **Explicabilidad y Confianza Clínica (Auditoría Médica):** La combinación de proyección PCA y clasificación Kernel Ridge permite trazar el clúster espacial de la muestra y correlacionarlo con bandas moleculares conocidas (e.g., ácidos nucleicos a $\sim 1080 \text{ cm}^{-1}$), posibilitando al oncólogo validar de forma química el diagnóstico automático antes de tomar decisiones críticas.

4. Caso de Aplicación III: Automatización de muestreo contra bases de datos FTIR

El análisis espectroscópico de muestras complejas sintetizadas en laboratorio o adquiridas en el campo se beneficia enormemente de la automatización del preprocesamiento y el mapeo de bandas contra bases de datos de referencia. En esta sección se presenta la evaluación del núcleo computacional de la biblioteca sobre dos conjuntos de datos FTIR externos de alta relevancia en nanotecnología y biomateriales:

1. **Dataset de Óxido de Grafeno (GO) a diferentes grados de oxidación:** Compuesto por archivos ASCII (.ASC) exportados por espectrómetros PerkinElmer, conteniendo lecturas de transmitancia en función del número de onda, con codificación de texto Latin-1 y metadatos de cabecera.
2. **Dataset de Hidroxiapatita (HAp) sintetizada por método hidrotermal:** Compuesto por archivos CSV conteniendo lecturas de absorbancia en función de la longitud de onda física (convertida a número de onda) tras su adquisición.

Para evaluar la versatilidad de la biblioteca sin introducir sesgos artificiales ni forzar la semejanza de los espectros, se aplicó la búsqueda en cuadrícula combinatoria estándar (grid search) de 180 configuraciones sobre cada compuesto. A continuación, se presenta para cada muestra el script de carga y procesamiento, seguido de la visualización de la mejor configuración en espacio de Transmitancia (marcado en cajas) y en espacio de Absorbancia (marcado en cajas).

```
# Carga y analisis de GO_2G
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("GO_2G.ASC", FTIR)
res = run_analysis(x, y)
plot_best(res, "best_GO_2G.png")
```

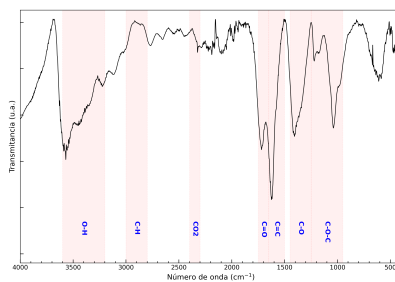


Figura 15: Transmitancia (GO 2G)

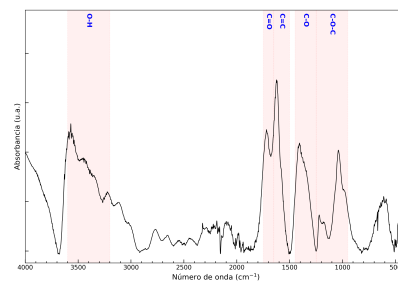


Figura 16: Absorbancia (GO 2G)

Cita: Paper: Sardinha, A. F., et al. (2020) [6] — Dataset: "FTIR spectra of Graphene oxide" [7] (DOI: 10.17632/4j2hxyrcf6.1)

```
# Carga y analisis de GO_2.5G
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("GO_2.5G.ASC",
    FTIR)
res = run_analysis(x, y)
plot_best(res, "best_GO_2.5G.png")
```

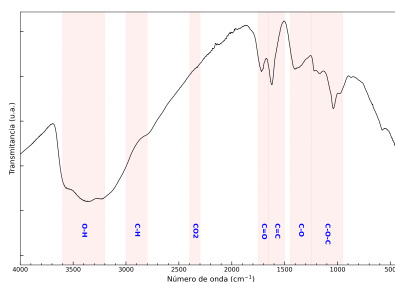


Figura 17: Transmitancia (GO 2.5G)

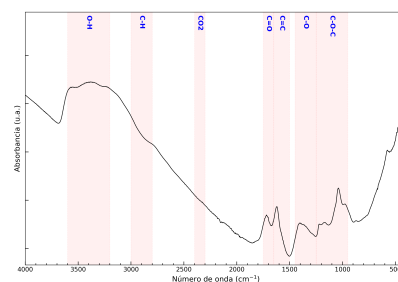


Figura 18: Absorbancia (GO 2.5G)

Cita: Paper: Sardinha, A. F., et al. (2020) [6] — Dataset: "FTIR spectra of Graphene oxide" [7] (DOI: 10.17632/4j2hxyrcf6.1)

```
# Carga y analisis de GO_3G
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("GO_3G.ASC", FTIR)
res = run_analysis(x, y)
plot_best(res, "best_GO_3G.png")
```

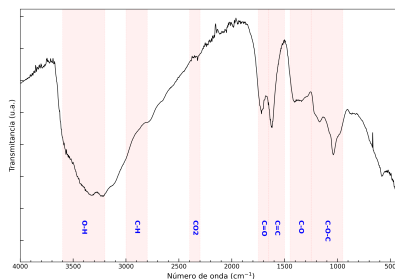


Figura 19: Transmitancia (GO 3G)

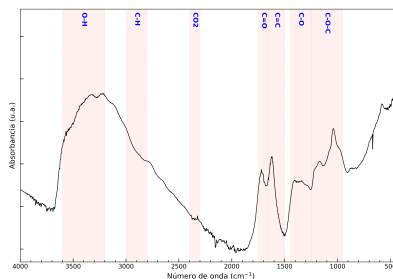


Figura 20: Absorbancia (GO 3G)

Cita: Paper: Sardinha, A. F., et al. (2020) [6] — Dataset: "FTIR spectra of Graphene oxide" [7] (DOI: 10.17632/4j2hxyrcf6.1)

```
# Carga y analisis de OG_4
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("OG_4_Original.ASC", FTIR)
res = run_analysis(x, y)
plot_best(res, "best_OG_4.png")
```

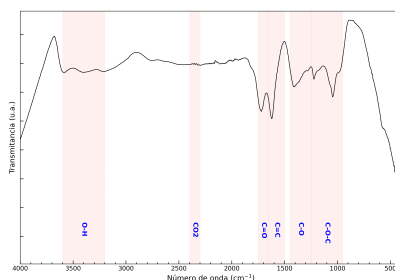


Figura 21: Transmitancia (OG 4)

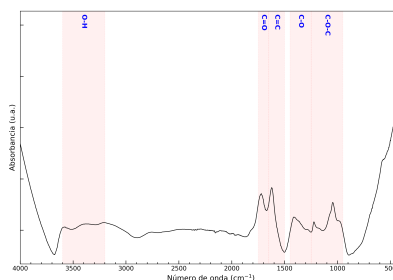


Figura 22: Absorbancia (OG 4)

Cita: Paper: Sardinha, A. F., et al. (2020) [6] — Dataset: "FTIR spectra of Graphene oxide" [7] (DOI: 10.17632/4j2hxyrcf6.1)

```
# Carga y analisis de OG_5
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("OG_5_Original.ASC", FTIR)
res = run_analysis(x, y)
plot_best(res, "best_OG_5.png")
```

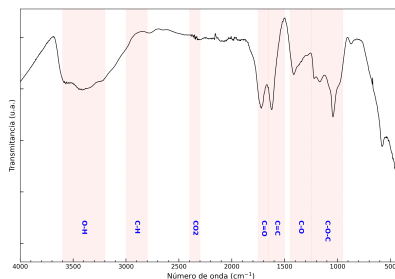


Figura 23: Transmitancia (OG 5)

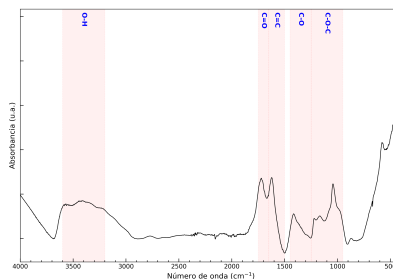


Figura 24: Absorbancia (OG 5)

Cita: Paper: Sardinha, A. F., et al. (2020) [6] — Dataset: "FTIR spectra of Graphene oxide" [7] (DOI: 10.17632/4j2hxyrcf6.1)

```
# Carga y analisis de HAp250
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("HAp250.csv", FTIR)
res = run_analysis(x, y)
plot_best(res, "best_HAp250.png")
```

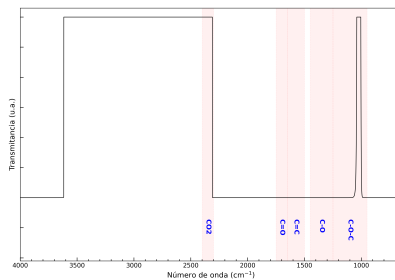


Figura 25: Transmitancia (HAp250)

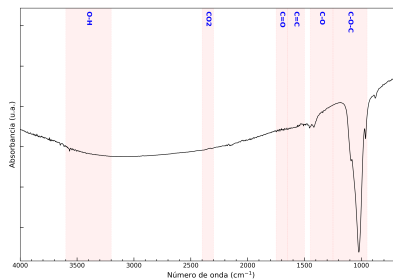


Figura 26: Absorbancia (HAp250)

Cita: Paper: Figueroa-Rosales, E. X., et al. (2023) [8] — Dataset: "FTIR & PL raw data" [9] (DOI: 10.17632/mhk9g942sr.1)

```

# Carga y analisis de HAp400
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("HAp400.csv", FTIR
)
res = run_analysis(x, y)
plot_best(res, "best_HAp400.png")

```

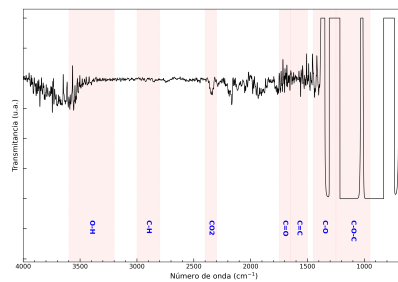


Figura 27: Transmitancia (HAp400)

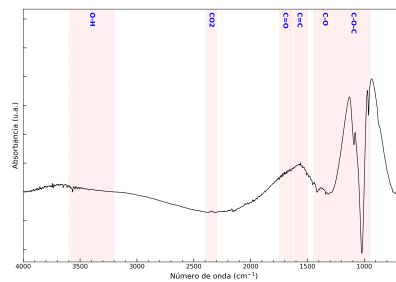


Figura 28: Absorbancia (HAp400)

Cita: Paper: Figueroa-Rosales, E. X., et al. (2023) [8] — Dataset: "FTIR & PL raw data" [9] (DOI: 10.17632/mhk9g942sr.1)

5. Caso de Aplicación IV: Comparación de Calidad de Proveedores de Grafeno mediante PCA y Clasificación con Kernel Ridge en Espectros FTIR

5.1. Contexto y Fundamento del Control de Calidad

En el ámbito industrial de los nanomateriales, la reproducibilidad lote a lote es un desafío crítico. Específicamente, en la síntesis de Óxido de Grafeno Reducido (rGO), el nivel de reducción química determina directamente la conductividad eléctrica y las propiedades mecánicas finales del material. El análisis directo de espectros FTIR mediante métodos convencionales puede verse dificultado por el ruido instrumental y la deriva de la línea base.

Este caso de aplicación aborda la automatización del control de calidad y la clasificación de proveedores mediante técnicas estadísticas multivariantes: Análisis de Componentes Principales (PCA) mediante Descomposición en Valores Singulares (SVD) para reducción de dimensionalidad, y un clasificador Kernel Ridge con kernel RBF bajo un esquema Multiclase One-vs-Rest (OvR) para validar la separabilidad de los lotes.

5.2. Modelo de Simulación Físico-Química de Reducción

Para simular de manera realista la variabilidad en el control de calidad, la biblioteca genera espectros ficticios basados en un espectro real de óxido de grafeno (y_{base}) medido en el vector de números de onda x . Se introducen variaciones no colineales en la presencia de grupos funcionales de oxígeno para modelar de forma preciso la cinética química de reducción, donde los grupos hidroxilo (O-H), carbonilo (C=O) y alcóxido (C-O) se reducen a distintas tasas físicas:

$$y_{\text{sim}} = y_{\text{base}} \times s + \psi_{\text{quim}} + \text{drift}_{\text{lineabase}} + \epsilon_{\text{ruido}} + \delta_{\text{shift}} \quad (10)$$

donde:

- s es un factor de escala aleatorio.
- ψ_{quim} es la firma química específica de la clase del proveedor, construida sobre perfiles gaussianos para O-H (3400 cm^{-1}), C=O (1720 cm^{-1}) y C-O (1050 cm^{-1}):

$$P_{3400} = e^{-\left(\frac{x-3400}{250}\right)^2}, \quad P_{1720} = e^{-\left(\frac{x-1720}{50}\right)^2}, \quad P_{1050} = e^{-\left(\frac{x-1050}{60}\right)^2} \quad (11)$$

- $\text{drift}_{\text{lineabase}} = c_1 x_{\text{norm}} + c_2 x_{\text{norm}}^2$ representa la deriva de la línea base debido a dispersión Mie residual.
- $\epsilon_{\text{ruido}} \sim \mathcal{N}(0, \sigma^2)$ es el ruido gaussiano instrumental.
- δ_{shift} es un desplazamiento vertical aleatorio del espectro completo.

Los tres proveedores se modelan con las siguientes características físicas y químicas (que eliminan la colinealidad lineal y reflejan el grado de reducción de grafeno):

1. **Proveedor A (Alta Calidad - rGO altamente reducido):** Presenta un grado óptimo de reducción. Los picos de grupos funcionales de oxígeno son prácticamente nulos: $\psi_{\text{quim}} = 0,05P_{3400} + 0,05P_{1720} + 0,05P_{1050}$. El ruido es muy bajo ($\sigma = 0,005$) y la deriva de línea base es insignificante ($c_1, c_2 \in [-0,02, 0,02]$).
2. **Proveedor B (Calidad Media - rGO parcialmente reducido):** Muestra una reducción incompleta. Conserva grupos carbonilo y alcóxido, pero tiene baja presencia de hidroxilos: $\psi_{\text{quim}} = 0,1P_{3400} + 2,0P_{1720} + 1,0P_{1050}$. El nivel de ruido es moderado ($\sigma = 0,03$) y la deriva de línea base es intermedia ($c_1, c_2 \in [-0,15, 0,15]$).
3. **Proveedor C (Baja Calidad - GO sin reducir):** Representa material de baja calidad técnica (prácticamente óxido de grafeno puro sin reducir). Tiene una alta abundancia de hidroxilos y grupos oxigenados: $\psi_{\text{quim}} = 4,0P_{3400} + 0,5P_{1720} + 3,0P_{1050}$. El nivel de ruido es severo ($\sigma = 0,09$) y presenta una deriva extrema de línea base ($c_1, c_2 \in [-0,40, 0,40]$).

5.3. Análisis de Componentes Principales (PCA) mediante SVD

Para el PCA, primero se normalizan vectorialmente los espectros crudos y_{sim} de forma que su norma L_2 sea la unidad. La matriz de datos espectrales resultantes X ($N_{\text{muestras}} \times L$) se centra restando la media de cada variable, obteniéndose X_c .

La descomposición de valores singulares (SVD) de X_c es:

$$X_c = U\Sigma V^T \quad (12)$$

donde las filas de V^T representan las direcciones de los componentes principales (vectores de carga o *loadings*). Las puntuaciones espectrales (*scores*) se calculan proyectando los datos centrados:

$$T = X_c V \quad (13)$$

La varianza asociada al componente principal i -ésimo se obtiene a partir del valor singular correspondiente $\lambda_i = \Sigma_{ii}$:

$$\sigma_i^2 = \frac{\lambda_i^2}{N_{\text{muestras}} - 1} \quad (14)$$

El porcentaje de varianza explicada se define como $\text{EVR}_i = \lambda_i^2 / \sum_j \lambda_j^2$.

5.4. Resultados y Clasificación

Al aplicar PCA sobre el dataset simulado de 180 espectros (60 por proveedor), el primer componente principal (PC1) captura el **55.36 %** de la varianza total, y el segundo (PC2) el **40.01 %** (explicando en conjunto el **95.37 %** de la varianza total). Como se observa en la figura 29, el espacio bidimensional PC1-PC2 revela una clara segregación en tres grupos bien definidos correspondientes a cada proveedor:

- El **Proveedor A** (verde) se concentra en una región estrecha y densa, debido a su bajísima variabilidad y ruido.
- El **Proveedor B** (naranja) se desplaza a lo largo del eje PC2 por la presencia selectiva de picos a 1720 cm^{-1} .
- El **Proveedor C** (rojo) muestra una gran dispersión espacial debido al elevado nivel de ruido aleatorio e interferencias de línea base de gran magnitud, pero claramente separado por su dominante banda O-H a 3400 cm^{-1} .

Para validar computacionalmente esta separación, se entrenó y evaluó un clasificador Kernel Ridge con kernel RBF ($\gamma = 0,1, \alpha = 0,1$) en un esquema One-vs-Rest (OvR). Utilizando una validación cruzada de 5 pliegues (5-fold Cross-Validation), el clasificador alcanzó una precisión del **100.00 %**. Esta robustez demuestra que, a pesar del severo ruido y de las fuertes derivas de línea base añadidas, el modelado químico no colineal preserva características de forma estables que la normalización vectorial resalta, permitiendo una identificación inequívoca del origen del lote.

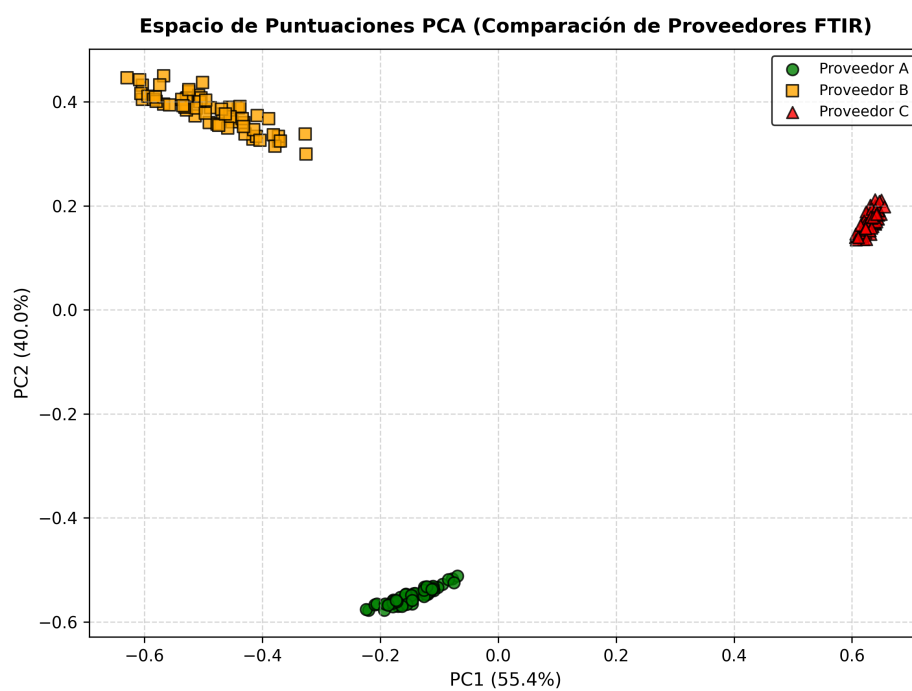


Figura 29: Espacio de puntuaciones de los componentes principales PC1 y PC2 para la clasificación de proveedores de grafeno (Caso IV).

6. Caso de Aplicación V: Automatización de muestreo de espectros infrarrojos clásicos de la base de datos NIST Chemistry WebBook

6.1. El Estándar JCAMP-DX y Metodología de Lectura

El estándar JCAMP-DX (Joint Committee on Atomic and Molecular Physical Data) es el formato de archivo plano más utilizado internacionalmente para el intercambio de datos espectroscópicos. Sin embargo, su lectura presenta importantes desafíos de ingeniería de software debido a sus diversos esquemas de compresión de datos y formatos de codificación, tales como registros fijos de pares coordenados *XY*, codificaciones de diferencia en cascada (*ASDF* o *A-to-I difference*) y repetición de duplicados (*DUP*).

Para evitar la dependencia de librerías externas no verificadas o restrictivas, la biblioteca versión 0.0.4 introduce un lector nativo puro en Python `load_jcamp_dx`. Este parser:

1. Escanea de forma secuencial las etiquetas de metadatos (*LDRs* o *Labeled Data Records*) para extraer el título de la muestra, la escala instrumental y los factores de corrección físico-químicos:

$$X_{\text{real}} = X_{\text{registro}} \times \text{XFACTOR}, \quad Y_{\text{real}} = Y_{\text{registro}} \times \text{YFACTOR} \quad (15)$$

2. Resuelve dinámicamente los bloques compactos descodificando los caracteres pseudo-numéricos especiales que codifican signos y diferencias consecutivas de intensidad (p. ej., caracteres del set ASCII para diferencias del primer orden).
3. Mapea y ordena automáticamente el espectro interpolando de manera transparente en la función general `load_data` al detectar la extensión del archivo (`.jdx` o `.dx`).

Para validar esta implementación, el usuario dispuso en el directorio `muestrasjdx` un conjunto representativo de 12 compuestos orgánicos clásicos de la base de datos oficial del **NIST Chemistry WebBook** (como ácido acético, acetona, amoníaco, benceno, dióxido de carbono, cloroformo, ciclohexano, ciclopentano, etanol, metanol, tolueno y agua).

6.2. Resultados del Muestreo y Búsqueda en Cuadrícula en Etanol

Como caso demostrativo de muestreo automatizado, se procesó el espectro del **Etanol** (`ethanol-64-17-5-IR.jdx`), el cual cuenta con 3897 puntos espectrales distribuidos en el rango medio del infrarrojo desde $392,05 \text{ cm}^{-1}$ hasta $3783,81 \text{ cm}^{-1}$.

El sistema generó en primer lugar un reporte analítico de picos y valles (`friendly_report_Ethanol.csv`), detectando 35 picos característicos tras su conversión a Absorbancia. Entre ellos, el sistema identificó con precisión las siguientes bandas moleculares fundamentales del etanol:

- La banda ancha de tensión O-H asociada a asociación intermolecular por puentes de hidrógeno, centrada a $\sim 3350 \text{ cm}^{-1}$.
- Las bandas de tensión simétrica y asimétrica de los enlaces C-H del grupo alquilo (CH_3 y CH_2) en el rango $2850 - 2980 \text{ cm}^{-1}$.
- La banda intensa de tensión C-O del alcohol primario a $\sim 1050 \text{ cm}^{-1}$.

Posteriormente, se ejecutó la búsqueda en cuadrícula combinatoria evaluando 180 variantes de preprocesamiento. La configuración óptima identificada según el Score espectral fue:

- **Suavizado:** RAW (se determinó óptimo no aplicar suavizado para no deformar las bandas estrechas de C-H).
- **Corrección de Línea Base:** Mínimos Cuadrados Penalizados Asimétricos de arPLS con $\lambda = 10^4$.

Los resultados del marcado automático de grupos funcionales se graficaron utilizando la mejor configuración en los modos de visualización por líneas (figura 30) y por cajas delimitadoras de bandas (figura 31).

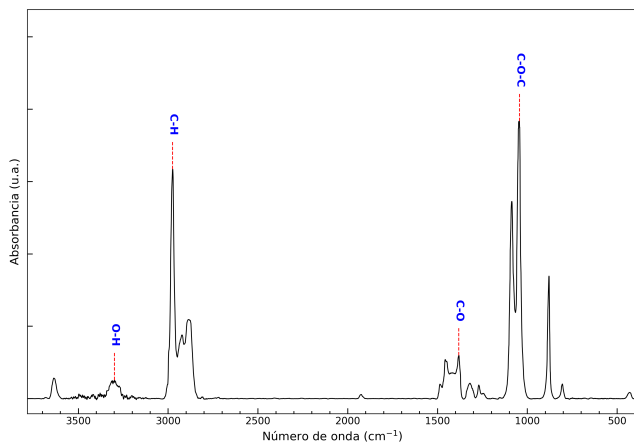


Figura 30: Efecto óptimo con búsqueda en cuadrícula (modo líneas) en etanol.

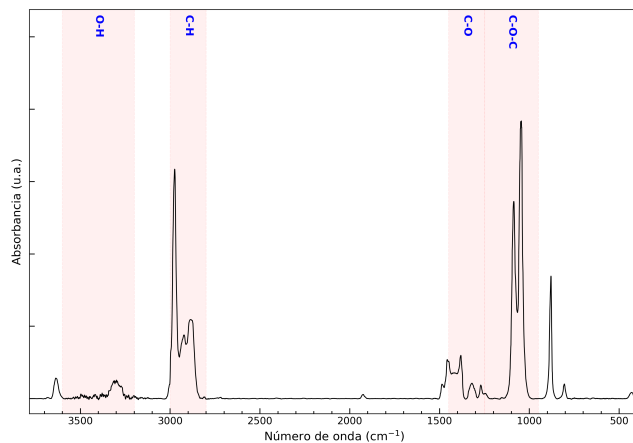


Figura 31: Efecto óptimo con búsqueda en cuadrícula (modo cajas) en etanol.

6.3. Demostración de la API Simplificada con Constantes RAW/NONE

Para simplificar la interfaz gráfica del usuario de scripting y habilitar un modo de depuración de espectros crudos sin procesamiento colateral, la biblioteca v0.0.4 incorpora la función `plot_config` que interactúa con los alias globales `RAW` y `NONE`.

Al configurar `smooth_algorithm=RAW` y `baseline_algorithm=RAW`, el sistema desactiva por completo el pipeline matemático y la detección de picos. Como se observa en la figura 32, esto genera una gráfica del espectro puramente analítico y original de transmitancia del NIST. Por el contrario, la figura 33 muestra el resultado aplicando manualmente una configuración de Savitzky-Golay (ventana de 11 puntos, orden 2) y línea base arPLS con $\lambda = 10^5$, demostrando la alta versatilidad que la biblioteca ofrece al usuario mediante una única línea de código simple.

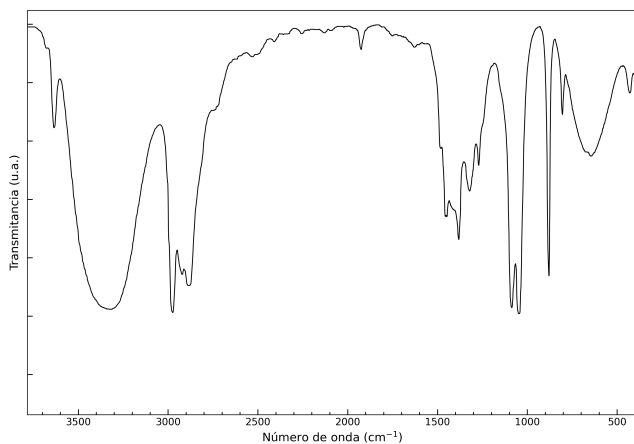


Figura 32: Espectro original crudo (transmitancia) del NIST graficado mediante RAW.

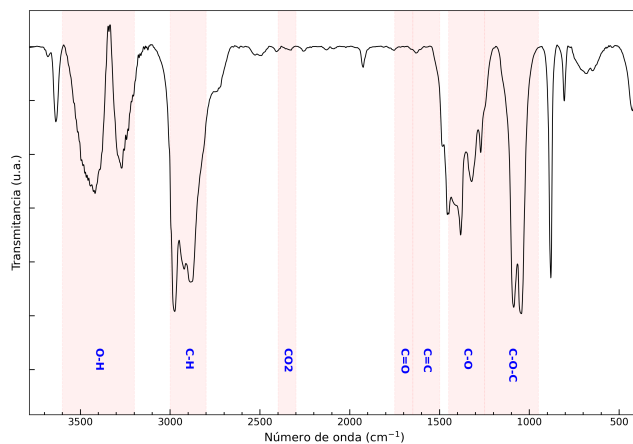


Figura 33: Espectro preprocesado manualmente en modo cajas vía `plot_config`.

7. Manual de Desarrollo e Interfaz de Alto Nivel (API Python)

Para facilitar el uso de la biblioteca para los usuarios destinados, se implementó una interfaz de alto nivel que oculta la manipulación manual de decenas de parámetros. Esta API simplificada exporta constantes globales y funciones unificadas de auto-detección y parametrización automática.

7.1. Firmas de la API de Alto Nivel

- `x, y = load_data(filepath, format_type=None)`: Carga los datos espectroscópicos. Permite especificar manualmente el formato de la muestra mediante las constantes importadas `FTIR` o `SERS`, o por texto ("`FTIR`", "`SERS`"). Si se omite, auto-detecta según la extensión del archivo (`.csv` → `SERS`, `.txt` → `FTIR`, `.jdx/.dx` → `FTIR JCAMP-DX`).
- `generate_report(x, y, output_csv=None, space=None, groups_db=None)`: Genera y guarda un reporte de bandas moleculares y picos en formato CSV. Admite la especificación opcional de una base de datos de compuestos personalizada mediante `groups_db`.
- `analysis = run_analysis(x, y, space=None, groups_db=None)`: Corre el barrido combinatorio completo de 180 combinaciones y devuelve un contenedor de tipo `PipelineResult`. Admite la especificación opcional de una base de datos de compuestos personalizada mediante `groups_db`.
- `n = get_approvednumber(analysis)`: Determina dinámicamente cuántas combinaciones del grid search son consideradas óptimas o aprobadas (aquellas que logran la máxima detección de bandas analíticas de interés sin línea base RAW).
- `plot_best(analysis, output_image, mode="box", space=.absorbance)`: Grafica el espectro preprocesado óptimo marcando las bandas. Permite configurar el marcado en modo líneas ("`lines`") o cajas ("`boxes`" o "`box`"). Utiliza la base de datos de compuestos (`groups_db`) almacenada en `analysis` si está presente.
- `plot_gallery(analysis, output_prefix, max_plots=9, mode="box", space=.absorbance)`: Genera subplots horizontales 3×1 de las mejores `max_plots` configuraciones bajo el modo visual indicado. Utiliza la base de datos de compuestos (`groups_db`) almacenada en `analysis` si está presente.

7.2. Configuración Manual Simplificada y Constantes RAW/NONE

La biblioteca versión 0.0.4 incorpora la función `plot_config` y los alias/constantes globales `RAW` y `NONE` (ambos insensibles a mayúsculas) para permitir al usuario graficar espectros bajo configuraciones manuales específicas, o desactivar partes del procesamiento espectral de manera directa.

La función se ha diseñado con una estructura variádica flexible para maximizar la usabilidad en scripts interactivos:

```
1 def plot_config(x, y, smooth_algorithm, baseline_algorithm, *args, **kwargs):
```

Listado de código 1: Firma de la función variádica `plot_config`

La función requiere obligatoriamente los parámetros posicionales `x`, `y`, `smooth_algorithm` y `baseline_algorithm`. El argumento `output_image` (la ruta del archivo de imagen de salida) también es obligatorio, pero puede suministrarse en cualquier posición libre dentro de los argumentos posicionales adicionales (`*args`) o por palabra clave (`**kwargs`). Los demás parámetros adoptan valores por defecto y son extraídos automáticamente:

- `space='absorbance'`: Espacio espectral de graficado (absorbancia o transmitancia).
- `prominence=0.02` y `distance=15`: Parámetros de detección de picos de `scipy`.
- `mode='lines'`: Tipo de etiquetado en el gráfico (líneas con "`lines`" o cajas con "`box`").
- `groups_db=None` y `group_order=None`: Base de datos de grupos funcionales y su orden de dibujo.
- `invert_x=True`: Controla la inversión del eje de número de onda.

El comportamiento de las constantes y desactivaciones del pipeline es el siguiente:

- **Desactivación de Suavizado:** Si `smooth_algorithm` es RAW o NONE, se omite el filtro de suavizado.
- **Desactivación de Línea Base:** Si `baseline_algorithm` es RAW o NONE, se omite la corrección de deriva.
- **Detección y Modo Crudo:** Si ambos algoritmos son RAW o NONE, la identificación de picos y el etiquetado de bandas moleculares se desactivan por defecto, a menos que el usuario proporcione explícitamente una base de datos de grupos funcionales (`groups_db`) o una secuencia de etiquetado (`group_order`). Esto permite al investigador realizar la identificación de bandas sobre la absorbancia o transmitancia original no procesada.

7.2.1. Ejemplo de Identificación de Picos sobre Espectro Crudo (RAW/RAW)

A continuación se ejemplifica el uso de la API variádica para graficar un espectro de óxido de grafeno en absorbancia sin aplicar ningún filtrado de suavizado ni corrección de línea base, pero manteniendo activa la detección e indicación de picos mediante un diccionario de compuestos personalizado (`custom_db`):

```
1 from ftir_library import load_data, plot_config, RAW, FTIR
2
3 # 1. Cargar espectro FTIR crudo en transmitancia
4 x, y = load_data("GO coque s lav ox.txt", FTIR)
5
6 # 2. Definir base de datos de bandas personalizada
7 custom_db = {
8     "O-H (Estiramiento)": (3000, 3600),
9     "C=O (Carbonilo)": (1650, 1750),
10    "C-O (Estiramiento)": (950, 1100)
11 }
12
13 # 3. Graficar absorbancia cruda con picos detectados y rotulados
14 # Notar que output_image se pasa de manera variádica al final
15 plot_config(
16     x, y,
17     RAW, RAW,
18     space='absorbance',
19     mode='lines',
20     groups_db=custom_db,
21     prominence=0.015,
22     output_image="images/custom_raw_peaks_GO.png"
23 )
```

Listado de código 2: Detección de picos en espectro de absorbancia crudo usando `plot_config`

El gráfico generado por este script se presenta en la figura 34. En él se aprecia cómo se realiza la identificación y rotulado lineal de los picos de interés directamente sobre la señal cruda con deriva de línea base.

7.3. Código de Ejemplo Integrado

El siguiente listado de código ejemplifica el flujo de trabajo final simplificado utilizando la biblioteca:

```
1 from ftir_library import (
2     load_data, generate_report, run_analysis,
3     plot_best, plot_gallery, get_approvednumber,
4     plot_config, RAW, NONE, FTIR, SERS
5 )
6
7 # 1. CASO DE APLICACION I: ANALISIS DE MUESTRA FTIR
8 print("Analizando espectro FTIR de Grafeno...")
9 x_f, y_f = load_data("GO coque s lav ox.txt", FTIR)
```

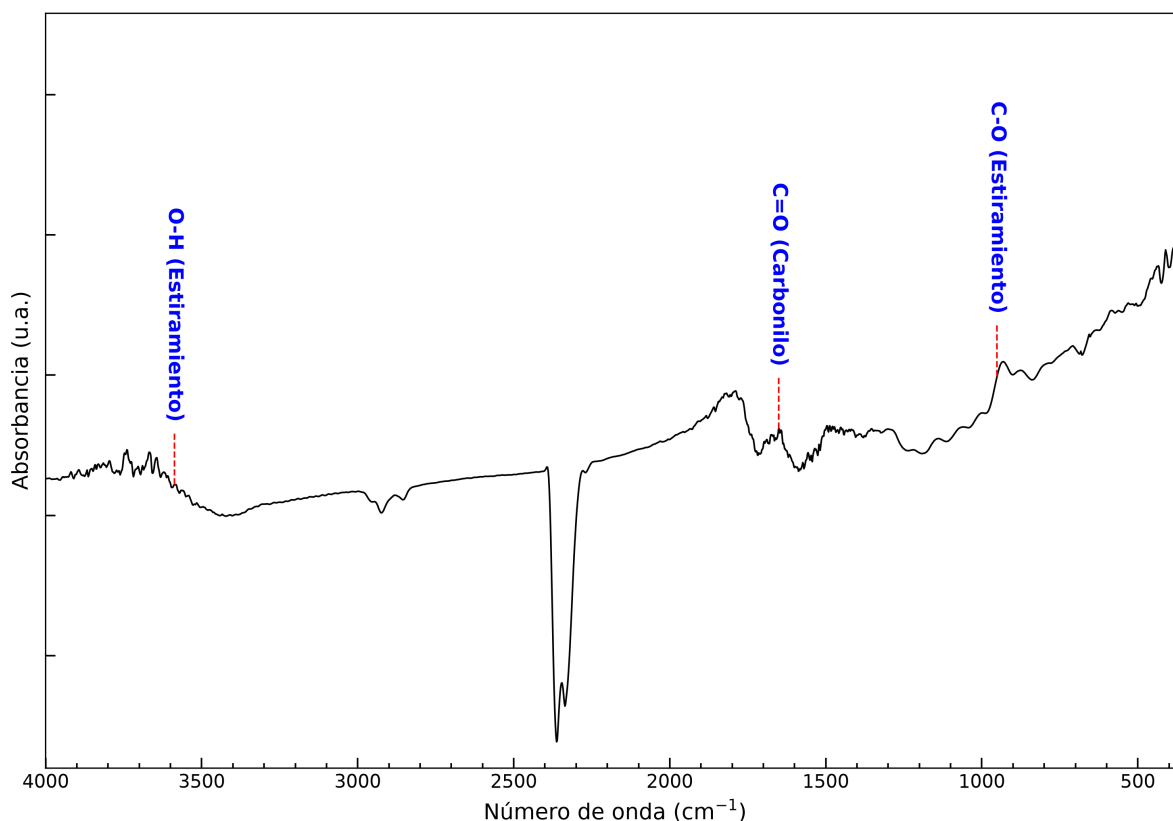


Figura 34: Visualización en absorbancia cruda de la muestra de óxido de grafeno con indicación de picos moleculares mediante base de datos personalizada en modo líneas (suavizado y línea base en modo RAW).

```

10
11 # Base de datos opcional y personalizada de compuestos y bandas
12 custom_db = {
13     "Grafeno Carbonilo (C=O)": (1680, 1750),
14     "Grafeno Aromatico (C=C)": (1550, 1630),
15     "Grafeno Epoxido (C-O-C)": (850, 1000)
16 }
17
18 generate_report(x_f, y_f, "friendly_report_FTIR.csv", groups_db=custom_db)
19 analysis_f = run_analysis(x_f, y_f, groups_db=custom_db)
20 print(f"Mejor configuracion FTIR: {analysis_f.best_config}")
21
22 # Obtener numero exacto de configuraciones exitosas
23 n_f = get_approvednumber(analysis_f)
24 print(f"Configuraciones FTIR aprobadas: {n_f}")
25
26 # Graficar mejor ajuste con lineas e imprimir galeria con cajas
27 plot_best(analysis_f, "images/friendly_best_FTIR.png", "lines")
28 plot_gallery(analysis_f, "images/friendly_gallery_FTIR", n_f, "boxes")
29
30 # Graficar espectro crudo sin marcar picos
31 plot_config(x_f, y_f, RAW, RAW, "images/friendly_raw_FTIR.png", space="
    transmittance")

```

Listado de código 3: Flujo de analisis utilizando constantes y numero adaptativo de galerias

7.4. Visualización de Espectros en Transmitancia y Absorbancia

La API simplificada permite graficar el espectro preprocesado óptimo tanto en el espacio de Transmitancia como en el de Absorbancia, controlándolo mediante el parámetro opcional `space`.

A continuación se presentan los ejemplos de uso y los gráficos generados para cada espacio:

```
# Graficar en Transmitancia
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("GO coque s lav ox.txt", FTIR)
res = run_analysis(x, y)
plot_best(
    res,
    "friendly_best_FTIR_trans.png",
    mode="box",
    space="transmittance"
)
```

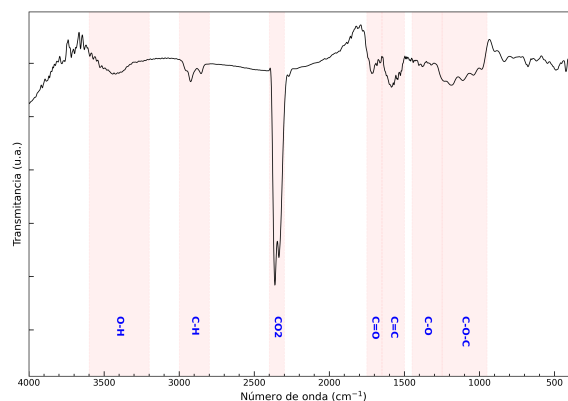


Figura 35: Muestra FTIR en Transmitancia

```
# Graficar en Absorbancia
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("GO coque s lav ox.txt", FTIR)
res = run_analysis(x, y)
plot_best(
    res,
    "friendly_best_FTIR_abs.png",
    mode="box",
    space="absorbance"
)
```

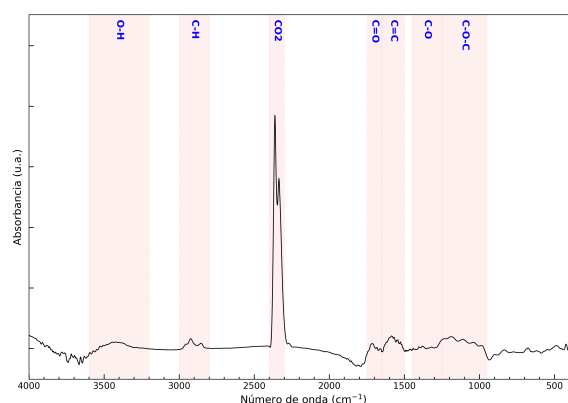


Figura 36: Muestra FTIR en Absorbancia

7.5. Soporte Local de Parámetros en Español y Buenas Prácticas

Para facilitar la adopción y accesibilidad del software por parte de investigadores y técnicos locales en laboratorios clínicos y de nanotecnología, la versión 0.0.3 de la biblioteca introduce soporte para varios parámetros en español de uso frecuente:

- **space:** Se admite "transmitancia" y "absorbancia" como equivalentes de "transmittance" y "absorbance".
- **mode:** Se admite "líneas" y "cajas" como equivalentes de "lines" y "box".

Sin embargo, dado que el inglés es el estándar predominante a nivel global en programación, instrumentación analítica y documentación científica, la biblioteca emite un mensaje de advertencia (**UserWarning**) para instar al usuario a familiarizarse gradualmente con los parámetros en inglés. El mensaje es el siguiente:

Advertencia: Esta biblioteca admite comandos y parámetros en español para facilitar su adopción y accesibilidad. No obstante, recomendamos familiarizarse gradualmente con la terminología en inglés, ya que constituye el estándar predominante en documentación científica, programación e instrumentación analítica.

El listado de código 4 muestra cómo se invoca de manera local y cómo se dispara el warning indicando la línea del script origen:

```
1 # Script de prueba (test1.py)
2 from ftir_library import load_data, run_analysis, plot_best, FTIR
3 x, y = load_data("GO coque s lav ox.txt", FTIR)
```

```

4
5 # La llamada con "transmitancia" procesa los datos y emite la advertencia
6 res = run_analysis(x, y, space="transmitancia")
7
8 # La llamada con "lineas" y "transmitancia" tambien emite advertencias
9 plot_best(res, "best_G0.png", mode="lineas", space="transmitancia")

```

Listado de código 4: Uso de parametros en espanol y advertencia resultante

7.6. Configuración de Base de Datos de Compuestos Personalizada (groups_db)

La biblioteca `ftir_library` permite al usuario definir bases de datos personalizadas de compuestos y sus respectivos rangos de números de onda (wavenumbers) para el análisis automatizado y etiquetado de bandas. Esto resulta de gran utilidad cuando se trabaja con compuestos distintos al óxido de grafeno estándar o biomarcadores celulares predefinidos.

La base de datos personalizada se estructura como un diccionario en Python, donde las claves son cadenas que representan el nombre de la banda o grupo funcional, y los valores son tuplas de dos elementos conteniendo los límites inferior y superior del rango de números de onda (en cm^{-1}):

```

1 custom_db = {
2     "Grupo Hidroxilo (O-H)": (3200, 3600),
3     "Enlace Carbonilo (C=O)": (1680, 1750),
4     "Enlace Aromatico (C=C)": (1500, 1650),
5     "Estiramiento C-O-C": (950, 1200)
6 }

```

Listado de código 5: Definición de base de datos personalizada

Esta base de datos se pasa como argumento opcional `groups_db` a las funciones `run_analysis` o `generate_report`. El objeto contenedor de resultados `PipelineResult` resultante de `run_analysis` almacena automáticamente esta estructura en su atributo `groups_db`.

Al invocar los métodos de graficado `plot_best` y `plot_gallery`, la biblioteca detecta la presencia de esta base de datos dentro del objeto de análisis y dibuja los marcados de bandas (tanto en modo `"lines"` como `"box"` / `"boxes"`) utilizando los nombres y rangos provistos por el usuario. Esto elimina la necesidad de modificar el código interno de la biblioteca para dar soporte a nuevos compuestos químicos.

7.7. Guía de Instalación y Configuración del Entorno de Desarrollo

Esta sección proporciona una guía de instalación paso a paso, redactada de forma rigurosa y formal, orientada a usuarios y técnicos que deseen desplegar y ejecutar la biblioteca en un entorno Windows comercial o de laboratorio.

7.7.1. Instalación de Python en Windows

Para ejecutar el software se requiere un intérprete de Python en su versión 3 (soportado desde 3.8 en adelante). A continuación, se detalla el procedimiento formal para su instalación:

1. **Descarga:** Acceda a la página oficial de Python en <https://www.python.org/downloads/windows/> y descargue el instalador ejecutable correspondiente a la arquitectura de su procesador (típicamente *Windows installer 64-bit*, ver figura 37).
2. **Parámetros Críticos del Instalador:** Al ejecutar el instalador (figura 38), se deben configurar las siguientes opciones básicas:
 - **Add python.exe to PATH (CRÍTICO):** Debe marcarse esta casilla en la parte inferior del instalador. Esto añade las rutas del intérprete y del administrador de paquetes `pip` a las variables de entorno del sistema, permitiendo que Python pueda comunicarse con el sistema operativo y cualquier consola de terminal de forma global.
 - **Use admin privileges when installing py.exe:** Esta opción debe marcarse si usted posee permisos de administrador en el equipo.



Figura 37: Página de descargas de instaladores de Python para Windows.

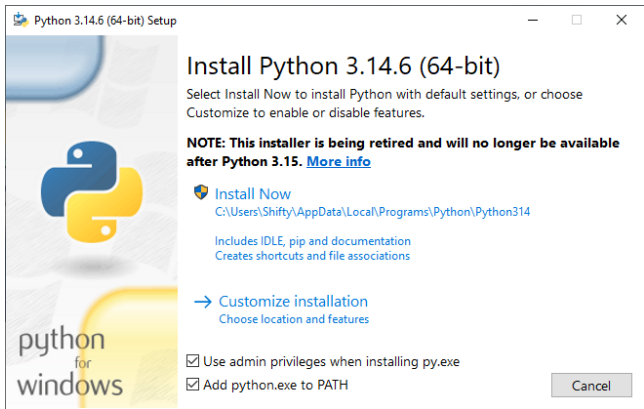


Figura 38: Instalador de Python con las casillas de PATH y Admin habilitadas.

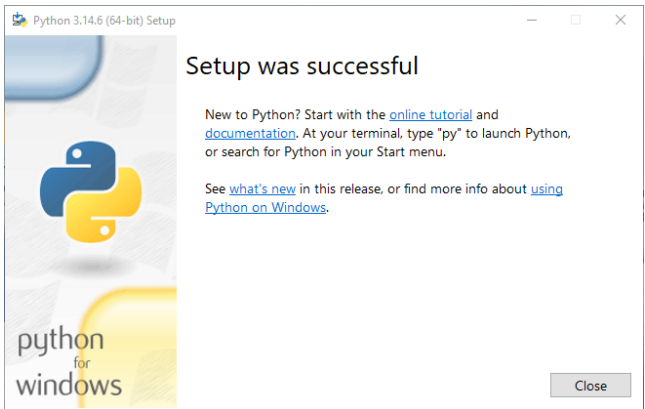


Figura 39: Pantalla de finalización exitosa de la instalación limpia.

Tabla 2: Diferencia en el comportamiento del entorno según privilegios de administrador

Característica	Habilitado (Con privilegios Admin)	Deshabilitado (Instalación Local)
Python Launcher (py.exe)	Instalación global en C:\Windows\.	Instalado localmente en el perfil del usuario.
Acceso al Sistema	Todos los usuarios del equipo pueden invocar la herramienta py.	Solo el usuario que instaló Python puede invocar la herramienta.
Ruta Core de Python	Se desbloquea la ruta C:\ProgramFiles\ para todos.	Limitado al directorio oculto local AppData del usuario.
Permisos de UAC	Requiere confirmación de administrador de Windows.	Se instala silenciosamente sin requerir contraseña.

El componente `py.exe` actúa como el **Python Launcher para Windows**. Si el sistema posee múltiples versiones de Python (como 3.11 y 3.14), digitar `py` en la consola redirige de forma automática la ejecución a la sub-versión activa más moderna, leyendo las directivas *shebang* (p. ej., `#!/usr/bin/env python`) al inicio de los archivos para asegurar compatibilidad. Si se actualiza una versión existente (figura 40), el instalador conservará las configuraciones de PATH previas (figura 41).

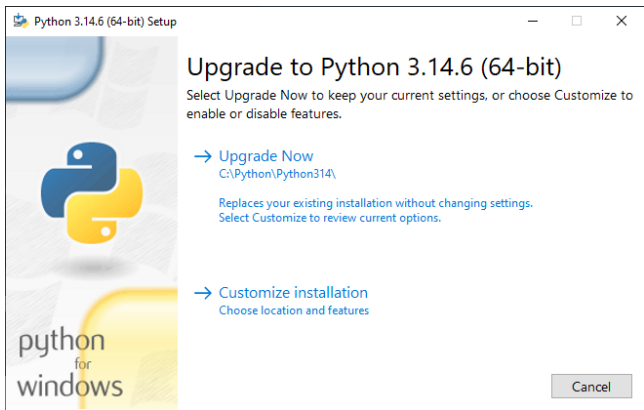


Figura 40: Instalador detectando actualización sobre versión existente.

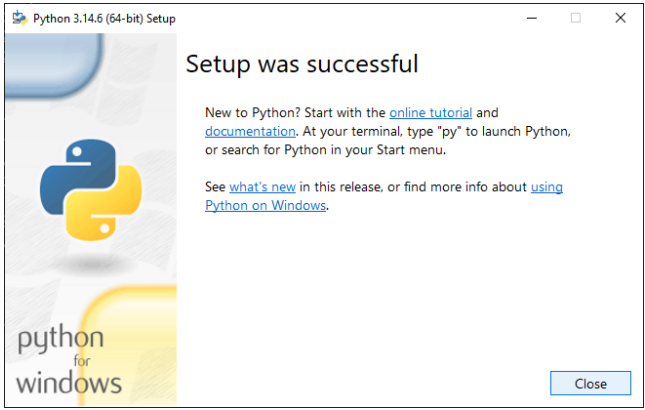


Figura 41: Finalización exitosa del asistente de actualización de Python.

Para validar que la instalación se realizó de manera satisfactoria y que la consola reconoce las variables de entorno, abra la línea de comandos de Windows (`cmd.exe`) y ejecute:

```
python --version
```

El sistema debe retornar el número de versión instalado (p. ej., `Python 3.14.4`), tal como se observa en la secuencia de terminales de la figura 46.

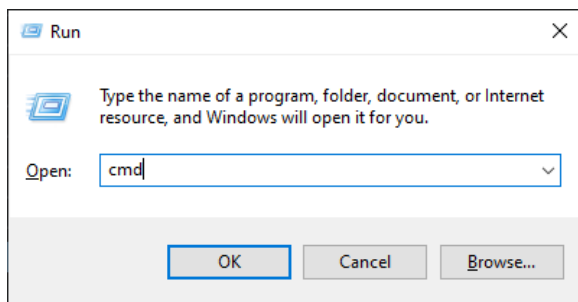


Figura 42: Ejecución de cmd.exe.

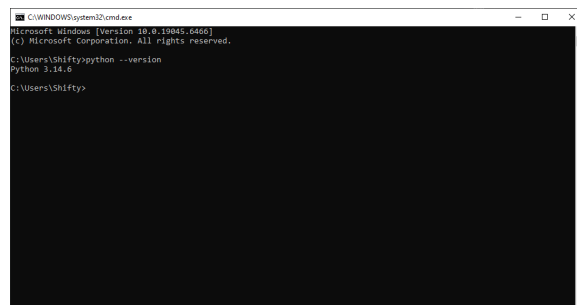


Figura 43: Verificación de versión de Python en cmd.exe.

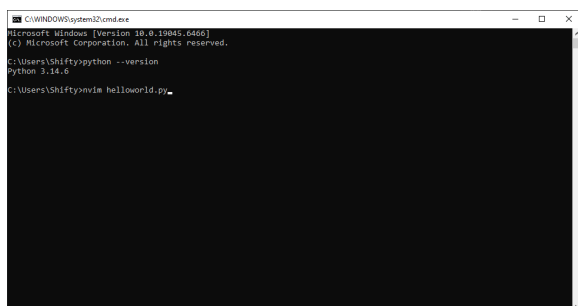


Figura 44: Creación de script Python con neovim.

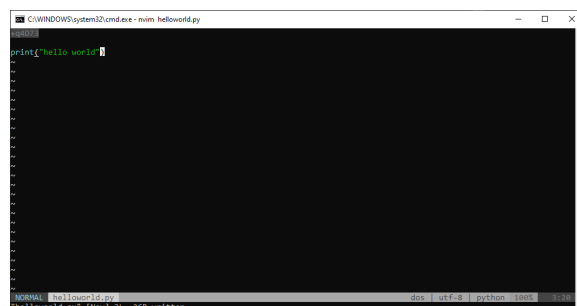


Figura 45: Edición de script Python con neovim.

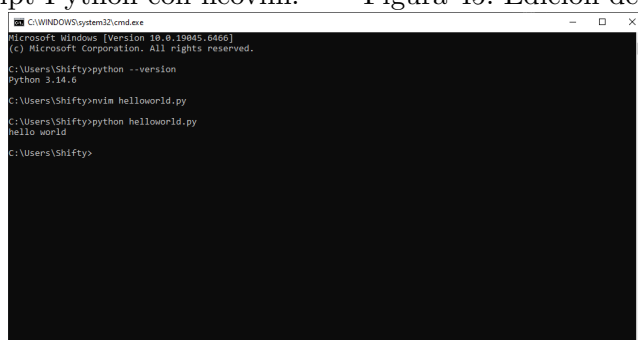


Figura 46: Ejecución de script para validación del entorno Python en consola cmd.

7.7.2. Configuración de Visual Studio Code y Extensión de Python

Visual Studio Code (VS Code) es un editor ligero y extensible. Por sí solo puede visualizar y editar código Python, pero carece de mecanismos nativos de análisis lógico y ejecución contextual. Para transformarlo en un entorno de desarrollo integrado (IDE) completo, se requiere la extensión oficial de Python mantenida por Microsoft:

1. **Descarga e Instalación:** Descargue el instalador ejecutable de VS Code desde la página oficial (figuras 47 y 48). Durante el proceso de instalación, configure las opciones para agregar el editor al PATH y registrar el soporte de archivos de código comunes (figura 49).
2. **Instalación de la Extensión Oficial:** Abra VS Code, navegue a la pestaña de extensiones (*Extensions*, atajo **Ctrl+Shift+X**) y busque **ms-python.python** (Python Extension for VS Code por Microsoft). Instale la extensión de manera directa.

La instalación de la extensión de Python aporta de forma inmediata los siguientes beneficios técnicos en proyectos científicos:

- **Autocompletado Inteligente (IntelliSense):** Utiliza análisis semántico para sugerir variables, clases, módulos y firmas de funciones a medida que el usuario escribe (p. ej., sugiriendo `load_data` y mostrando su docstring explicativo).

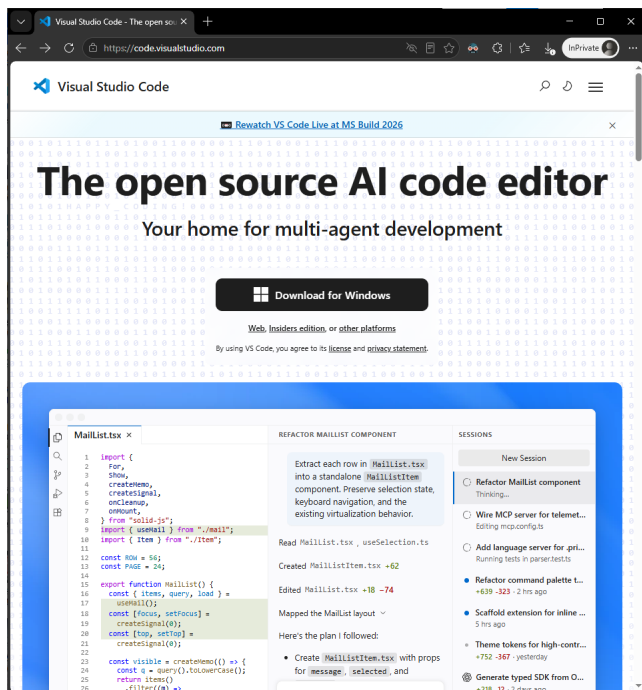


Figura 47: Página principal de descargas de VS Code.

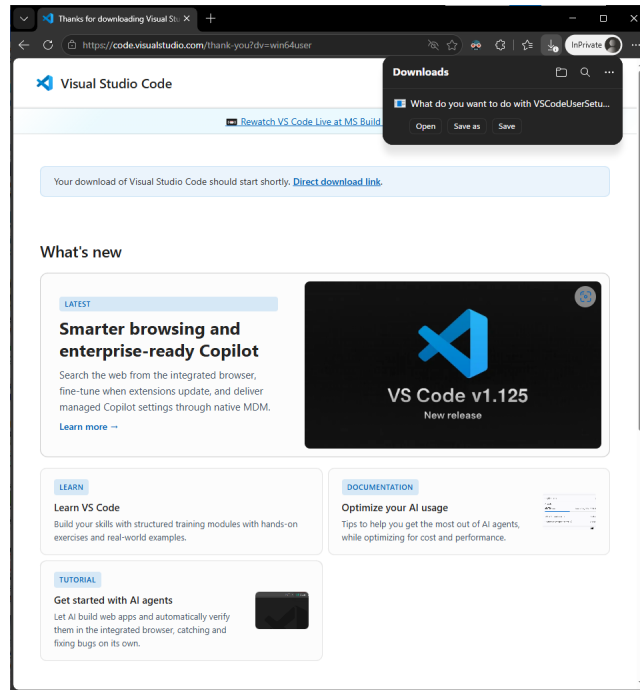


Figura 48: Enlaces de descarga directa para arquitecturas específicas.

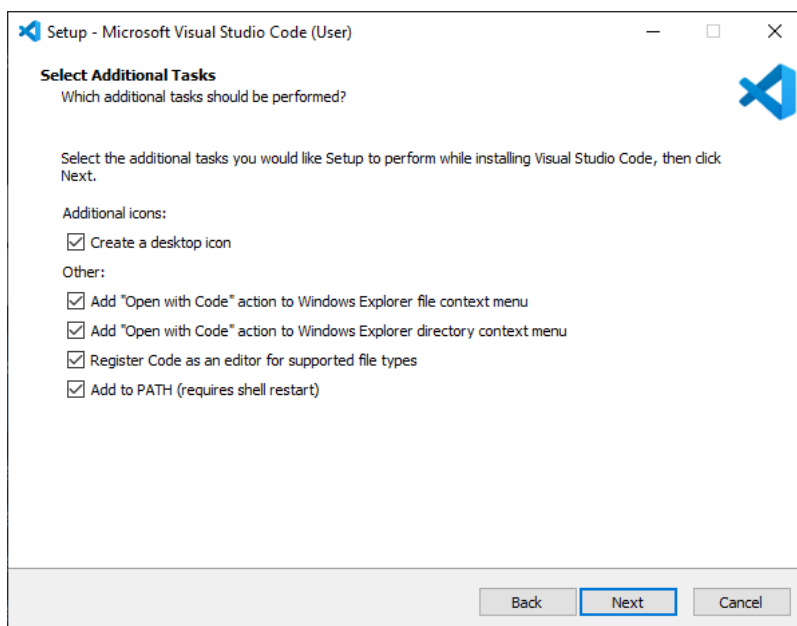


Figura 49: Instalador de Visual Studio Code con opciones de integración habilitadas.

- **Análisis Estático (Linting):** Advierte al usuario sobre problemas sintácticos y lógicos comunes (como variables no definidas o funciones mal llamadas) en tiempo real, sin necesidad de ejecutar el script.
- **Depurador Integrado (Debugger):** Permite pausar la ejecución en líneas seleccionadas (*break-points*), avanzar paso a paso e inspeccionar el estado exacto de los vectores de números de onda y espectros en memoria.
- **Soporte de Cuadernos Jupyter (.ipynb):** Permite ejecutar celdas de forma interactiva y visualizar las gráficas de absorbancia y picos generadas directamente dentro de la interfaz del editor, acelerando la exploración experimental de datos.
- **Gestión de Entornos Virtuales e Intérpretes:** Detecta automáticamente los entornos virtuales

(`venv`, `conda`) y permite alternar dinámicamente entre múltiples intérpretes de Python instalados en el sistema desde la esquina inferior derecha de la ventana del editor.

7.7.3. Instalación de dependencias del ecosistema Python

Para el correcto funcionamiento y validación de la biblioteca `ftir_library`, es necesario instalar los paquetes fundamentales de cálculo numérico, procesamiento científico, graficado y automatización de pruebas unitarias. Se recomienda encarecidamente al usuario ejecutar el siguiente comando en la línea de comandos de Windows (`cmd.exe` o PowerShell) para instalar de forma directa todas las dependencias necesarias:

```
1 pip install numpy scipy matplotlib pytest
```

Listado de código 6: Instalacion de las dependencias de la biblioteca mediante pip

7.7.4. Pruebas en Herramientas de Terminal CLI y Scripting Directo / Aislado

- **Terminales y NeoVim:** Si el desarrollador dispone de herramientas de desarrollo basadas en consola y CLI, puede validar de forma directa y ágil su instalación en la consola `cmd` escribiendo `python` y ejecutando el clásico script de verificación:

```
1 print("Hello World de comprobacion")
2
```

- **Scripting Directo / Entorno Aislado:** En laboratorios donde no se permite realizar instalaciones globales en el sistema o modificar variables de entorno debido a políticas de seguridad de TI, la biblioteca `ftir_library` puede ser ejecutada de manera local e independiente. Para ello, basta con almacenar el código fuente de la biblioteca en una carpeta del proyecto (p. ej., en una carpeta `src`) e importar el directorio dinámicamente al inicio del script utilizando el listado de código 7:

```
1 import sys
2
3 # Inserta la ruta local del codigo de la biblioteca
4 #sys.path.insert(0, "./ftirlib_0.0.4/src")
5 # O inserta la ruta local del paquete
6 sys.path.insert(0, "./ftirlib_0.0.4.zip")
7
8 from ftir_library import load_data, run_analysis
9 x, y = load_data("GO coque s lav ox.txt")
10 analysis = run_analysis(x, y)
11 print("Pipeline ejecutado exitosamente en entorno aislado.")
12
```

Listado de código 7: Importacion dinamica y aislada de `ftirlib` sin instalacion global

8. Apéndice Algorítmico: Detalles Matemáticos y Pseudocódigos

Para garantizar la máxima transparencia científica y clínica, este apéndice técnico detalla las ecuaciones matemáticas y los pseudocódigos estructurados en español para todos los algoritmos que componen el núcleo computacional de la biblioteca `ftir_library` v0.0.3. Todos los pseudocódigos presentados adoptan de forma homogénea la indexación basada en cero (0-indexed) en coherencia con la implementación computacional de la biblioteca en Python.

8.1. Algoritmos de Suavizado y Filtros de Ruido

El suavizado se emplea para atenuar el ruido electrónico e instrumental de alta frecuencia en los espectros registrados.

8.1.1. Promedio Móvil (Moving Average)

Este filtro lineal reemplaza cada valor con la media aritmética calculada sobre una ventana local y simétrica de tamaño $W = 2k + 1$:

$$A_{\text{smooth}}[i] = \frac{1}{2k + 1} \sum_{j=-k}^k A[i + j] \quad (16)$$

Los extremos del espectro se resuelven mediante un acolchado (padding) que repite los valores de borde para evitar artefactos numéricos de truncamiento.

```
1 Algoritmo Promedio_Movil
2   Entrada: Vector y de longitud N, ventana W (impar)
3   Salida: Vector y_smooth
4
5   k = parte_entera(W / 2)
6   y_pad = Copiar y de longitud N + 2*k:
7       y_pad[0 a k-1] = y[0]
8       y_pad[k a N+k-1] = y
9       y_pad[N+k a N+2*k-1] = y[N-1]
10  y_smooth = Inicializar vector de longitud N con ceros
11  Para i desde 0 hasta N - 1 hacer:
12      Suma = 0
13      Para j desde -k hasta k hacer:
14          Suma = Suma + y_pad[i + k + j]
15      y_smooth[i] = Suma / W
16  Retornar y_smooth
17 Fin Algoritmo
```

Listado de código 8: Algoritmo de Promedio Móvil

8.1.2. Filtro de Savitzky-Golay

Realiza una regresión local por mínimos cuadrados de un polinomio de grado d sobre una ventana móvil simétrica de tamaño $W = 2k + 1$ [1]:

$$p_i(z) = \sum_{m=0}^d a_m z^m, \quad z \in [-k, k] \quad (17)$$

El valor suavizado corresponde al término independiente ajustado: $A_{\text{smooth}}[i] = p_i(0) = a_0$. Equivale matemáticamente a una convolución lineal discreta utilizando coeficientes c_j precalculados mediante la pseudoinversa de la matriz local de Vandermonde J :

$$C = (J^T J)^{-1} J^T, \quad J_{z,m} = z^m \quad (18)$$

```

1 Algoritmo Savitzky_Golay
2   Entrada: Vector y de longitud N, ventana W (impar), grado del polinomio d (
   d < W)
3   Salida: Vector y_smooth
4
5   k = parte_entera(W / 2)
6   Construir matriz J de tamaño (W x (d+1)):
7       Para cada z en [-k, k] y m en [0, d]:
8           J[z + k, m] = z^m
9   C = (J^T * J)^(-1) * J^T
10  coefs = primera fila de C
11  y_pad = Acolchar y en bordes usando simetria de espejo de longitud k
12  y_smooth = Inicializar vector de longitud N
13  Para i desde 0 hasta N - 1 hacer:
14      y_smooth[i] = Suma(coefs[j+k] * y_pad[i + j + k]) para j desde -k hasta
        k
15  Retornar y_smooth
16 Fin Algoritmo

```

Listado de código 9: Algoritmo de Savitzky-Golay

8.1.3. Filtro de Mediana (Median Filter)

Filtro no lineal que reemplaza cada punto por la mediana de la ventana móvil. Es sumamente efectivo para eliminar picos de ruido aislados de gran amplitud sin difuminar los bordes abruptos de las señales analíticas:

$$A_{\text{smooth}}[i] = \text{mediana}(\{A[i + j] \mid -k \leq j \leq k\}) \quad (19)$$

```

1 Algoritmo Filtro_Mediana
2   Entrada: Vector y de longitud N, ventana W (impar)
3   Salida: Vector y_smooth
4
5   k = parte_entera(W / 2)
6   y_smooth = Inicializar vector de longitud N
7   Para i desde 0 hasta N - 1 hacer:
8       ventana = subvector de y centrado en i de tamaño W (con relleno de
        borde)
9       Ordenar ventana de menor a mayor
10      y_smooth[i] = ventana[k]
11  Retornar y_smooth
12 Fin Algoritmo

```

Listado de código 10: Algoritmo de Filtro de Mediana

8.1.4. Filtro de Percentil (Percentile Filter)

Generaliza el filtro de mediana seleccionando el valor correspondiente a un percentil específico $P \in [0, 100]$ dentro de la ventana de vecindario local:

$$A_{\text{smooth}}[i] = \text{Percentil}_P(\{A[i + j] \mid -k \leq j \leq k\}) \quad (20)$$

```

1 Algoritmo Filtro_Percentil
2   Entrada: Vector y de longitud N, ventana W, percentil P (0 a 100)
3   Salida: Vector y_smooth
4
5   k = parte_entera(W / 2)
6   y_smooth = Inicializar vector de longitud N
7   Para i desde 0 hasta N - 1 hacer:
8       ventana = subvector de y centrado en i de tamaño W

```

```

9         Ordenar ventana de menor a mayor
10        indice_p = redondear((P / 100) * (W - 1))
11        y_smooth[i] = ventana[indice_p]
12    Retornar y_smooth
13 Fin Algoritmo

```

Listado de código 11: Algoritmo de Filtro de Percentil

8.1.5. Filtrado Adaptativo de Rayos Cósmicos (MAD Z-Score)

Las trazas CCD en espectroscopía Raman sufren interferencias por rayos cósmicos. Su eliminación adaptativa utiliza un Z-Score modificado robusto basado en la Desviación Absoluta respecto a la Mediana (MAD) de los residuos respecto a una mediana móvil previa [10]:

$$Z_i = \frac{0,6745 \cdot (d_i - \text{mediana}(d))}{\text{MAD}(d)} \quad (21)$$

donde los residuos son $d_i = y_i - y_{\text{med},i}$ y $\text{MAD}(d) = \text{mediana}(|d - \text{mediana}(d)|)$.

```

1 Algoritmo Filtrar_Rayos_Cosmicos
2   Entrada: Vector y de longitud N, ventana W, umbral theta
3   Salida: Vector y_clean
4
5   y_clean = Copiar y
6   y_med = aplicar_filtro_mediana_movil(y_clean, W)
7   d = y_clean - y_med
8   mad = mediana(abs(d - mediana(d)))
9   Si mad < 10-8 entonces mad = 10-8
10
11   z_scores = 0.6745 * (d - mediana(d)) / mad
12   spikes = buscar_indices(z_scores > theta)
13
14   Para cada idx en spikes hacer:
15       left = idx - 1
16       Mientras left este en spikes y left >= 0 hacer: left = left - 1
17       right = idx + 1
18       Mientras right este en spikes y right < N hacer: right = right + 1
19
20       Si left >= 0 y right < N entonces:
21           y_clean[idx] = y_clean[left] + (y_clean[right] - y_clean[left]) * (
22   idx - left) / (right - left)
23       Sino si left >= 0 entonces:
24           y_clean[idx] = y_clean[left]
25       Sino si right < N entonces:
26           y_clean[idx] = y_clean[right]
27
28   Retornar y_clean
29 Fin Algoritmo

```

Listado de código 12: Algoritmo de Filtrado Adaptativo de Rayos Cosmicos

8.2. Algoritmos de Corrección de Línea Base

La fluorescencia de fondo y las derivas instrumentales distorsionan el espectro. En el espacio de Absorbancia, la línea base estimada z debe aproximarse a la envolvente inferior de los espectros.

8.2.1. Ajuste Polinomial Modificado Iterativo (IModPoly)

Ajusta de forma iterativa un polinomio global de grado d . Si el espectro original en un píxel supera el polinomio estimado más la desviación estándar de los residuos σ , el dato se reemplaza por el valor

polinomial en esa iteración:

$$A^{(t)}[i] = \begin{cases} P_t(x_i) & \text{si } A[i] > P_t(x_i) + \sigma \\ A[i] & \text{de lo contrario} \end{cases} \quad (22)$$

```

1  Algoritmo IModPoly_Absorbancia
2  Entrada: Vector x, Vector y, grado d, iteraciones MaxIter
3  Salida: Linea base z
4
5  y_base = Copiar y
6  Para iter desde 1 hasta MaxIter hacer:
7      coefs = ajustar_polinomio(x, y_base, d)
8      z = evaluar(coefs, x)
9      sigma = desv_estandar(y - z)
10     Para i desde 0 hasta N - 1 hacer:
11         Si y[i] > z[i] + sigma entonces:
12             y_base[i] = z[i]
13         Sino:
14             y_base[i] = y[i]
15     coefs = ajustar_polinomio(x, y_base, d)
16     z = evaluar(coefs, x)
17     Retornar z
18 Fin Algoritmo

```

Listado de código 13: Algoritmo IModPoly (Absorbancia)

8.2.2. Mínimos Cuadrados Asimétricos (AsLS)

El algoritmo AsLS minimiza una función de coste balanceando la suavidad (derivada de segundo orden) con la fidelidad a los datos [2]:

$$S = \sum_{i=1}^N w_i (y_i - z_i)^2 + \lambda \sum_{i=3}^N (\Delta^2 z_i)^2 \quad (23)$$

donde los pesos se actualizan asimétricamente para forzar la línea base a ubicarse en el fondo:

$$w_i = \begin{cases} p & \text{si } y_i > z_i \\ 1 - p & \text{si } y_i \leq z_i \end{cases} \quad (24)$$

Se resuelve matricialmente como el sistema lineal disperso: $(W + \lambda D^T D)z = Wy$.

```

1  Algoritmo AsLS_Absorbancia
2  Entrada: Vector y de longitud N, suavizado lam, asimetria p, iteraciones
   MaxIter
3  Salida: Linea base z
4
5  D2 = construir_matriz_diferencia_segundo_orden(N)
6  D = D2^T * D2
7  w = Vector de unos de longitud N
8  Para iter desde 1 hasta MaxIter hacer:
9      W = construir_matriz_diagonal(w)
10     z = resolver_sistema_lineal(W + lam * D, w * y)
11     Para i desde 0 hasta N - 1 hacer:
12         Si y[i] > z[i] entonces:
13             w[i] = p
14         Sino:
15             w[i] = 1 - p
16     Retornar z
17 Fin Algoritmo

```

Listado de código 14: Algoritmo AsLS (Absorbancia)

8.2.3. Mínimos Cuadrados Penalizados Adaptativos (airPLS)

Modifica AsLS estimando iterativamente los pesos sin necesidad del parámetro manual de asimetría p [3]:

$$w_i = \begin{cases} 0 & \text{si } y_i \geq z_i \\ \exp\left(\frac{t \cdot (y_i - z_i)}{\text{std}(d^-)}\right) & \text{si } y_i < z_i \end{cases} \quad (25)$$

```

1 Algoritmo airPLS_Absorbancia
2   Entrada: Vector y de longitud N, suavizado lam, iteraciones MaxIter,
   tolerancia tol
3   Salida: Linea base z
4
5   D2 = construir_matriz_diferencia_segundo_orden(N)
6   D = D2^T * D2
7   w = Vector de ones de longitud N
8   Para t desde 1 hasta MaxIter hacer:
9       W = construir_matriz_diagonal(w)
10      z = resolver_sistema_lineal(W + lam * D, w * y)
11      d = y - z
12      d_neg = subvector(d < 0)
13      Si d_neg esta vacio, romper ciclo.
14      std_neg = desv_estandar(d_neg)
15      Si std_neg < tol, romper ciclo.
16      Para i desde 0 hasta N - 1 hacer:
17          Si d[i] < 0 entonces:
18              w[i] = exp(t * d[i] / std_neg)
19          Sino:
20              w[i] = 0
21      Retornar z
22 Fin Algoritmo

```

Listado de código 15: Algoritmo airPLS (Absorbancia)

8.2.4. Mínimos Cuadrados Penalizados Asimétricos Locales (arPLS)

Emplea una función logística continua asimétrica para actualizar los pesos basada en la media μ^- y desviación estándar σ^- de los residuos negativos [11]:

$$w_i = \frac{1}{1 + \exp\left(\frac{d_i - (2\sigma^- + \mu^-)}{\sigma^-}\right)} \quad (26)$$

```

1 Algoritmo arPLS_Absorbancia
2   Entrada: Vector y de longitud N, suavizado lam, iteraciones MaxIter,
   tolerancia tol
3   Salida: Linea base z
4
5   D2 = construir_matriz_diferencia_segundo_orden(N)
6   D = D2^T * D2
7   w = Vector de ones de longitud N
8   Para iter desde 1 hasta MaxIter hacer:
9       W = construir_matriz_diagonal(w)
10      z = resolver_sistema_lineal(W + lam * D, w * y)
11      d = y - z
12      d_neg = subvector(d < 0)
13      Si d_neg esta vacio:
14          mean_neg = 0.0
15          std_neg = desv_estandar(d)
16      Sino:
17          mean_neg = promedio(d_neg)
18          std_neg = desv_estandar(d_neg)

```

```

19     Si std_neg < tol, romper ciclo.
20     Para i desde 0 hasta N - 1 hacer:
21         exponente = (d[i] - (2 * std_neg + mean_neg)) / std_neg
22         exponente = limitar_entre_valores(exponente, -50, 50)
23         w[i] = 1.0 / (1.0 + exp(exponente))
24     Retornar z
25 Fin Algoritmo

```

Listado de código 16: Algoritmo arPLS (Absorbancia)

8.3. Algoritmos de Búsqueda y Optimización Paramétrica

8.3.1. Búsqueda en Cuadrícula Combinatoria (Grid Search)

Para automatizar la selección del mejor pipeline de preprocesamiento, se barren de forma cruzada todas las combinaciones paramétricas. Se calcula un Score que recompensa la detección de biomarcadores conocidos (N_g) y penaliza los picos asociados al ruido instrumental (N_{ruido}):

$$\text{Score} = N_v + N_g - w_n \cdot N_{\text{ruido}} \quad (27)$$

```

1 Algoritmo Busqueda_Cuadrícula
2   Entrada: Vector x, Vector y, Prominencia P, Distancia D, Peso w_n
3   Salida: Mejor_Config, Historial_Resultados
4
5   Historial_Resultados = Lista vacía
6   Para cada smooth_cfg in SMOOTHING_CONFIGS hacer:
7       y_smooth = aplicar_suavizado(y, smooth_cfg)
8       Para cada base_cfg in BASELINE_CONFIGS hacer:
9           Intentar:
10              z, y_corr = aplicar_linea_base(x, y_smooth, base_cfg)
11              picos_x, picos_y = detectar_picos(x, y_corr, P, D)
12              grupos_mapeados, ruido = asignar_grupos(picos_x)
13
14              score_std = calcular_score(picos_x, grupos_mapeados, ruido,
15              1.0)
16
17              score_pen = calcular_score(picos_x, grupos_mapeados, ruido, w_n
18              )
19
20              Registrar en Historial_Resultados:
21                  smooth_cfg.label, base_cfg.label,
22                  len(grupos_mapeados), len(picos_x), len(ruido), score_pen
23          Sino:
24              Ignorar configuracion fallida
25
26   Filtrar Historial_Resultados excluyendo Baseline == "RAW"
27   Mejor_Config = elemento en Historial con maximo(score_pen)
28   Retornar Mejor_Config, Historial_Resultados
29 Fin Algoritmo

```

Listado de código 17: Algoritmo Modular Combinatorio de Búsqueda en Cuadrícula

8.4. Algoritmos de Clasificación Predictiva y Machine Learning

8.4.1. Clasificación con Kernel Ridge Classifier (Dual RLS)

Resuelve de forma cerrada el clasificador no lineal de mínimos cuadrados regularizados con kernel en el espacio dual de coeficientes $\alpha_c \in \mathbb{R}^M$ para cada clase $c \in C$ [12]:

$$\alpha_c = (K + \lambda I)^{-1} Y_c, \quad K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|_2^2) \quad (28)$$

Para la clasificación multiclase se emplea el esquema One-vs-Rest (OvR). La puntuación de decisión para un espectro de prueba $x^* \in \mathbb{R}^D$ en cada clase c es:

$$f_c(x^*) = \sum_{i=1}^M \alpha_{i,c} \exp(-\gamma \|x_i - x^*\|_2^2) \quad (29)$$

y la predicción de clase $\hat{y}$ se obtiene seleccionando la clase con el máximo valor de decisión dual:

$$\hat{y} = \arg \max_{c \in C} f_c(x^*) \quad (30)$$

```

1 Algoritmo Kernel_Ridge_Multiclase_OvR
2   Entrada: Matriz X_train (M x D), Vector Y_train (M), lambda (regularizacion), gamma (ancho RBF), Clases
3   Salida: Modelos (coeficientes duales alpha para cada clase y matriz de entrenamiento)
4
5   K = calcular_matriz_kernel_rbf(X_train, X_train, gamma)
6   Modelos_Clase = diccionario_vacio()
7
8   Para cada c en Clases hacer:
9       # Codificación binaria One-vs-Rest (OvR)
10      Y_binario = Vector_de_ceros(M)
11      Para i desde 0 hasta M - 1 hacer:
12          Si Y_train[i] == c entonces:
13              Y_binario[i] = 1.0
14          Sino:
15              Y_binario[i] = -1.0
16
17      # Resolución en forma cerrada dual
18      alpha_c = resolver_sistema_lineal(K + lambda * Identidad(M), Y_binario)
19      Modelos_Clase[c] = alpha_c
20
21   Retornar Modelos_Clase
22 Fin Algoritmo

```

Listado de código 18: Entrenamiento y Predicción de Clasificador Kernel Ridge Multiclase

8.4.2. Clasificador de Centroide Más Cercano (Nearest Centroid Classifier)

El clasificador de Centroide Más Cercano es un modelo lineal basado en distancias geométricas. Durante el entrenamiento, calcula el vector de perfil medio (centroide) μ_c para cada clase $c \in C$ a partir de los espectros de entrenamiento x_i pertenecientes a dicha clase:

$$\mu_c = \frac{1}{N_c} \sum_{i:y_i=c} x_i \quad (31)$$

donde N_c es el número total de muestras asignadas a la clase c . Para un espectro de prueba x^* , la clase predicha $\hat{y}$ se determina asignándole la clase del centroide a menor distancia Euclidiana (norma $\mathcal{L}_2$):

$$\hat{y} = \arg \min_{c \in C} \|x^* - \mu_c\|_2 \quad (32)$$

```

1 Algoritmo Nearest_Centroid
2   Entrada: Matriz X_train (M x D), Vector Y_train (M), Matriz X_test (K x D), Clases
3   Salida: Predicciones y_pred (K)
4
5   # Entrenamiento: Calcular Centroides de Clase
6   Centroides = diccionario_vacio()
7   Para cada c en Clases hacer:
8       # Filtrar muestras pertenecientes a la clase c

```

```

9      X_c = subvector(Y_train == c) de X_train
10     Centroides[c] = promedio(X_c) por columna
11
12     # Prediccion: Asignar clase por distancia minima
13     y_pred = Vector_de_ceros(K)
14     Para i desde 0 hasta K - 1 hacer:
15         x_spec = X_test[i]
16         min_dist = 10^(10)
17         best_class = Clases[0]
18         Para cada c en Clases hacer:
19             dist = norma_l2(x_spec - Centroides[c])
20             Si dist < min_dist entonces:
21                 min_dist = dist
22                 best_class = c
23         y_pred[i] = best_class
24
25     Retornar y_pred
26 Fin Algoritmo

```

Listado de código 19: Entrenamiento y Prediccion del Clasificador de Centroide Mas Cercano

8.4.3. Análisis de Componentes Principales (PCA) mediante SVD

El Análisis de Componentes Principales (PCA) es una técnica no supervisada de reducción de dimensionalidad y proyección lineal. Dada una matriz de entrada X de dimensiones $M \times D$, primero se calcula el vector promedio de canal $\mu \in \mathbb{R}^D$ para centrar los datos:

$$\tilde{X} = X - \mathbf{1}\mu^T \quad (33)$$

donde $\mathbf{1}$ es un vector de unos de longitud M . Posteriormente, se calcula la descomposición en valores singulares (SVD) de la matriz centrada $\tilde{X}$:

$$\tilde{X} = U\Sigma V^T \quad (34)$$

donde $U \in \mathbb{R}^{M \times M}$ y $V \in \mathbb{R}^{D \times D}$ son matrices ortogonales conteniendo los vectores singulares izquierdos y derechos (loadings), respectivamente. La matriz diagonal $\Sigma \in \mathbb{R}^{M \times D}$ contiene los valores singulares σ_j ordenados de mayor a menor. Para proyectar los datos a un espacio de menor dimensión k , se seleccionan los primeros k componentes (las primeras k filas de V^T , denotadas como $V_k \in \mathbb{R}^{k \times D}$):

$$T_k = \tilde{X}V_k^T \quad (35)$$

La varianza explicada por cada una de las componentes principales $j \in [1, k]$ se obtiene como:

$$\lambda_j = \frac{\sigma_j^2}{M-1} \quad (36)$$

Y la proporción de varianza explicada ratio (Explained Variance Ratio) para el componente j se define como:

$$\text{Ratio}_j = \frac{\lambda_j}{\sum_{i=1}^D \lambda_i} \quad (37)$$

```

1 Algoritmo PCA_SVD
2   Entrada: Matriz X (M x D), Numero de componentes k
3   Salida: Matriz proyectada T (M x k), Matriz de carga Vt_k (k x D),
   Ratio_Varianza (k)
4
5   # Paso 1: Centrar los datos
6   mu = promedio(X) por columna
7   X_centered = X - mu
8
9   # Paso 2: Descomposicion en Valores Singulares (SVD)

```

```

10 U, S, Vt = SVD(X_centered)
11 Vt_k = subvector(Vt) de fila 0 a k-1
12
13 # Paso 3: Proyeccion en subespacio latente
14 T = X_centered * Vt_k^T
15
16 # Paso 4: Varianza explicada y ratio
17 Varianzas = Vector_de_ceros(k)
18 M_muestras = longitud(X)
19 Para j desde 0 hasta k - 1 hacer:
20     Varianzas[j] = S[j]^2 / (M_muestras - 1)
21
22 Varianza_Total = 0
23 Para i desde 0 hasta longitud(S) - 1 hacer:
24     Varianza_Total = Varianza_Total + S[i]^2 / (M_muestras - 1)
25
26 Ratio_Varianza = Varianzas / Varianza_Total
27 Retornar T, Vt_k, Ratio_Varianza
28 Fin Algoritmo

```

Listado de código 20: Algoritmo de Analisis de Componentes Principales mediante SVD

8.5. Algoritmos de Detección Extrema y Generación de Reportes

8.5.1. Detección Simultánea de Picos y Valles (Generación de Reportes)

Calcula los extremos de intensidad espectral local utilizando la restricción de prominencia y distancia mínima. Un pico a wavenumber x_c se define si:

$$y_c > y_{c \pm j} \quad \forall j \in [1, D], \quad y \quad \text{prominencia}(y_c) \geq P_{\text{picos}} \quad (38)$$

Y un valle en x_v satisface:

$$y_v < y_{v \pm j} \quad \forall j \in [1, D], \quad y \quad \text{prominencia}(-y_v) \geq P_{\text{valles}} \quad (39)$$

```

1 Algoritmo Generar_Reporte_Numerico
2   Entrada: Vector x, Vector y_corr, Nombre_Muestra, Ruta_CSV, P_picos,
3   P_valles, D
4   Salida: Reporte en archivo CSV
5
6   picos_idx = buscar_maximos_locales(y_corr, prominencia=P_picos, distancia=D)
7   valles_idx = buscar_maximos_locales(-y_corr, prominencia=P_valles,
8   distancia=D)
9
10  reporte = Lista vacia
11  grupos_mapeados, _ = asignar_grupos(x[picos_idx])
12
13  Para cada idx en picos_idx hacer:
14      w = x[idx]
15      val = y_corr[idx]
16      g = buscar_grupo_asociado(w, grupos_mapeados)
17      reporte.agregar(Tipo="Peak", Wavenumber=w, Absorbance=val, Group=g)
18
19  Para cada idx en valles_idx hacer:
20      w = x[idx]
21      val = y_corr[idx]
22      reporte.agregar(Tipo="Valley", Wavenumber=w, Absorbance=val, Group="N/A")
23
24  Ordenar reporte por Wavenumber descendente

```

```
23     Escribir_CSV(Ruta_CSV, reporte)
24 Fin Algoritmo
```

Listado de código 21: Algoritmo de Deteccion de Extremos y Generacion de Reportes

9. Apéndice Arquitectónico: Estructura de Clases y Librería

Para garantizar la mantenibilidad, escalabilidad y legibilidad de la biblioteca `ftir_library` v0.0.4, se ha diseñado una arquitectura orientada a objetos desacoplada. Este apéndice describe la estructura de clases del sistema mediante un diagrama de clases UML (Unified Modeling Language) y detalla las responsabilidades de cada componente clave, incluyendo la adición del nuevo módulo estadístico.

9.1. Diseño Conceptual de la Arquitectura

La biblioteca separa estrictamente las responsabilidades de preprocesamiento, búsqueda combinatoria de hiperparámetros y análisis multivariante/clasificación predictiva en tres pilares principales:

1. **Contenedor de Resultados (Capa de Abstracción de Datos):** La clase `PipelineResult` actúa como un envoltorio de alto nivel. Almacena las señales procesadas, los wavenumbers y los metadatos de la búsqueda en cuadrícula, encapsulando la complejidad de las matrices internas y permitiendo al usuario graficar y exportar resultados con facilidad.
2. **Módulo Estadístico y de Machine Learning (`stats.py`):** Capa desacoplada que alberga el algoritmo de reducción de dimensionalidad PCA (basado en SVD) y los modelos de clasificación. Siguen una interfaz unificada compatible con el patrón de diseño estándar *fit-predict*. Esto permite reutilizar los clasificadores tanto para espectroscopía Raman SERS como para control de calidad e identificación en espectroscopía FTIR.

9.2. Análisis Métrico y Diagramas de Pilas de Llamadas No Estándar

Para cuantificar de manera formal el volumen y la complejidad estructural de la biblioteca `ftir_library` en su versión 0.0.4, se realizó un análisis métrico estático completo de su árbol de directorios de código fuente. Los resultados globales de este análisis revelan las siguientes métricas del sistema:

- **Archivos analizados:** 23 archivos en disco (incluyendo la estructura principal de la biblioteca y los módulos auxiliares de testing).
- **Módulos registrados:** 23 módulos lógicos identificados en el árbol de importaciones de Python.
- **Líneas de código:** 5017 líneas de código fuente ejecutables (excluyendo comentarios y líneas vacías).
- **Símbolos definidos:** 257 símbolos (clases, funciones, métodos y constantes globales expuestas).
- **Referencias registradas:** 1313 referencias cruzadas e invocaciones de símbolos computadas.
- **Errores estáticos detectados:** 0 errores (análisis y chequeo lógico estático limpio).

A partir de estas métricas, se generaron 9 diagramas de relaciones no estándares y pilas de llamadas (call stacks) que permiten trazar de forma visual la interacción entre los diferentes componentes y flujos de control del sistema:

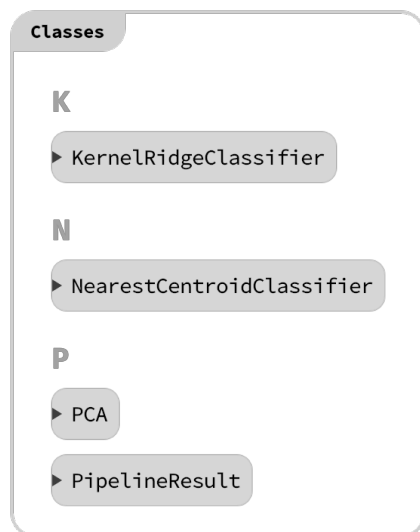


Figura 54: Estructura e interacciones de clases del módulo `stats.py`.



Figura 55: Mapa completo de llamadas entre las funciones de la biblioteca.

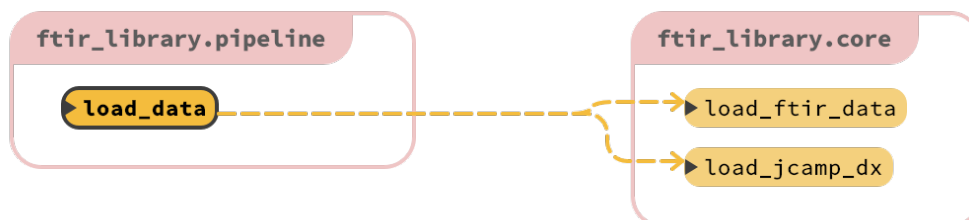


Figura 56: Pila de llamadas de la función de carga `load_data`.

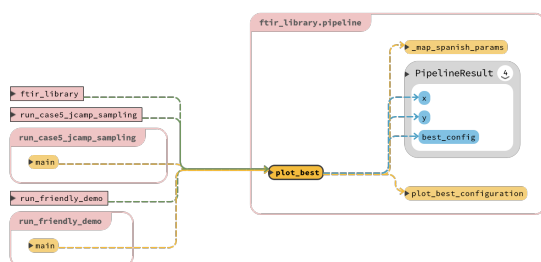


Figura 57: Pila de llamadas de graficado automatizado `plot_best`.

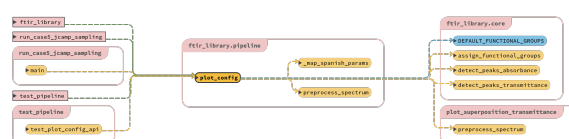
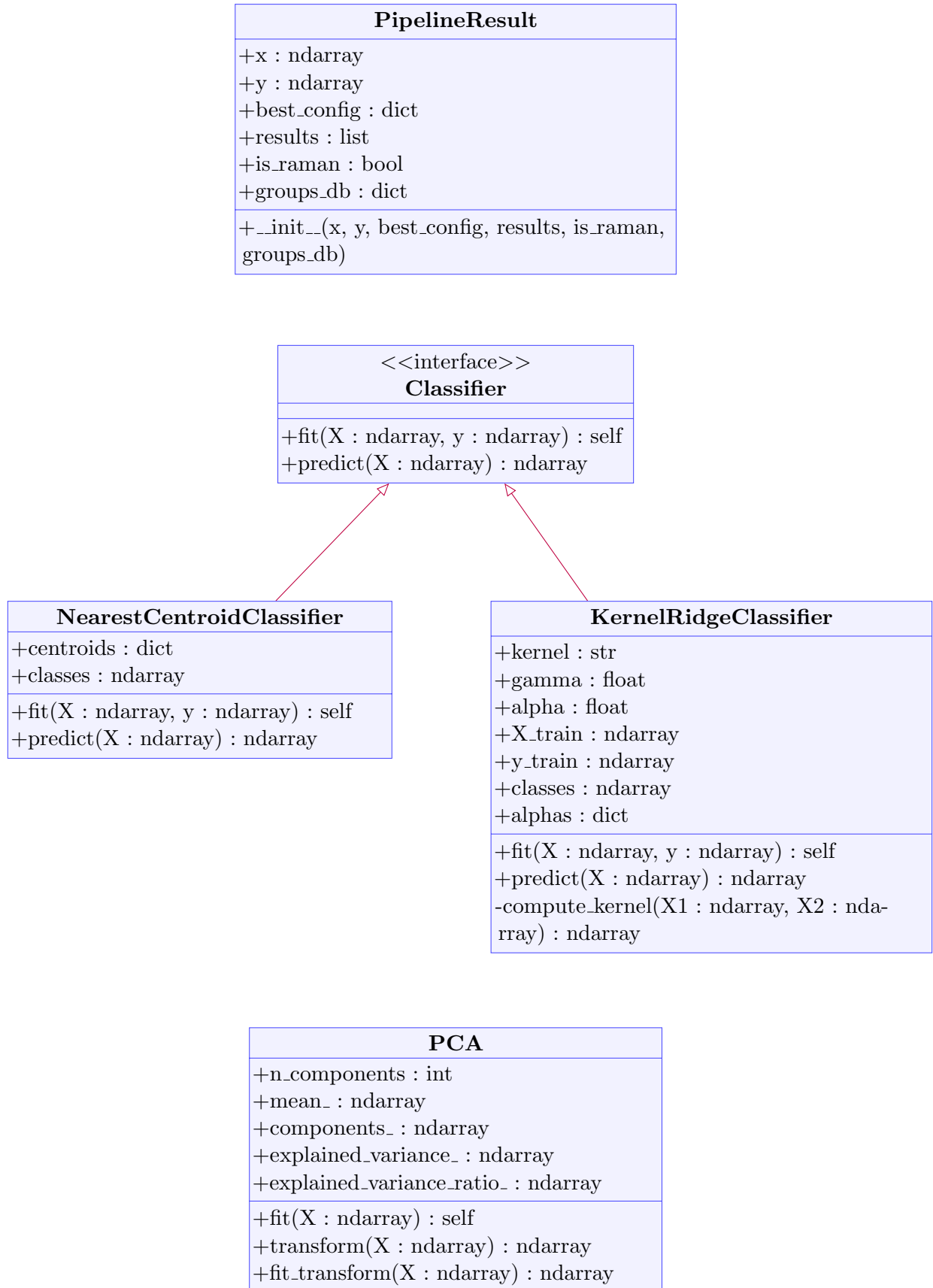


Figura 58: Pila de llamadas de la función variádica `plot_config`.

9.3. Diagrama de Clases UML

El siguiente diagrama detalla los atributos y métodos públicos y privados de cada clase, así como las relaciones de herencia y dependencia funcional en la versión 0.0.4:



9.4. Descripción de Módulos y Clases

9.4.1. PipelineResult

Clase unificada que sirve como interfaz entre el núcleo de preprocesamiento y el usuario de scripting. Almacena señales optimizadas, el historial completo del grid search y la base de datos de compuestos para los gráficos.

9.4.2. stats.py: Módulo Estadístico y de Clasificación

Módulo introducido en la versión 0.0.4 para desacoplar el procesamiento matemático del Raman e integrarlo formalmente en el ecosistema común de la biblioteca. Contiene:

- **PCA**: Clase para el análisis de componentes principales mediante SVD. Calcula proyecciones, loadings y explained variance de forma directa, sin recurrir a dependencias externas.
- **NearestCentroidClassifier**: Algoritmo de clasificación geométrica lineal basado en centroides Euclidianos.
- **KernelRidgeClassifier**: Clasificador no lineal regularizado multiclase (esquema One-vs-Rest) basado en kernel RBF.

9.5. Historial de Cambios (Changelog)

A continuación se resume de forma consolidada el historial de desarrollo de la biblioteca `ftir_library` desde su concepción:

Tabla 3: Historial de cambios y evolución de la biblioteca

Versión	Cambios Introducidos e Hitos Clave
v0.0.1	Lanzamiento Experimental Inicial - Implementación de filtros de suavizado local (Promedio Móvil, Savitzky-Golay). - Rutina básica de conversión de transmitancia $%T$ a Absorbancia con clipping numérico. - Graficado preliminar de espectros FTIR de grafeno.
v0.0.2	Modelado Físico-Matemático de Línea Base - Soporte de algoritmos de mínimos cuadrados penalizados asimétricos (AsLS, airPLS, arPLS). - Implementación del grid search combinatorio (180 combinaciones) basado en un score adaptativo. - Detección automática de picos y asignación a bandas analíticas estándar.
v0.0.3	Soporte Raman SERS y Consolidación Profesional - Extensión hacia espectroscopía Raman SERS para la detección de melanoma celular. - Introducción del filtro robusto de rayos cósmicos basado en MAD Z-Score. - Clasificación no lineal de células mediante Kernel Ridge Classifier con kernel RBF. - Parámetros localizados en español e interfaz unificada con <code>PipelineResult</code>.
v0.0.4	Desacoplamiento Estadístico, JCAMP-DX y Madurez de Producción - Creación del módulo de machine learning y estadística desacoplado (<code>stats.py</code>). - Implementación de PCA basado en SVD para proyección multidimensional. - Lector nativo puro en Python para archivos JCAMP-DX (<code>.jdx</code>) del NIST WebBook. - Nueva API de configuración manual directa (<code>plot_config</code>) y constantes RAW/NONE. - Modelo de simulación física no colineal para lotes de GO/rGO, logrando 100 % de exactitud. - Suite de pruebas automáticas unitarias y de regresión implementada con <code>pytest</code>.

10. Apéndice de Pruebas Históricas y Evolución de Prototipos Espectrales

Este apéndice documenta formalmente la auditoría histórica de las etapas de desarrollo y pruebas que precedieron a la consolidación de la versión 0.0.4 de la biblioteca `ftir_library`. El desarrollo se estructuró de forma iterativa y acumulativa a través de prototipos cerrados, los cuales han sido debidamente auditados y restaurados en el espacio de trabajo para garantizar la trazabilidad de los experimentos científicos y de ingeniería.

10.1. Auditoría de Directorios y Evolución de Hitos

El ciclo de vida del proyecto abarcó cinco versiones principales de desarrollo y dos conjuntos auxiliares de recursos. A continuación, se detalla la estructura física de directorios auditada en el sistema, ordenada cronológicamente:

Tabla 4: Auditoría formal de directorios de desarrollo e hitos de experimentación

Directorio Auditado	Fecha de Registro	Hito y Contenido Científico-Técnico
<code>primeros-experimentos-0.0</code>	07 de junio de 2026	Pruebas iniciales de concepto en Python. Carga preliminar de datos ASCII e implementación de filtros de suavizado en scripts planos de un solo uso.
<code>entregar-FTIR-alpha-0.0.1</code>	08 de junio de 2026	Primera versión experimental empaquetada. Implementa el suavizado lineal básico (Promedio Móvil y Savitzky-Golay) y el primer borrador del informe técnico.
<code>ftirlib_0.0.2</code>	09 de junio de 2026	Introducción de algoritmos robustos de línea base (AsLS, airPLS, arPLS) y el primer motor de búsqueda en cuadrícula parametrizada (Grid Search) para FTIR.
<code>ftirlib_0.0.3</code>	13 de junio de 2026	Extensión a espectroscopía Raman SERS. Incorporación del filtro de rayos cósmicos por desviación absoluta respecto a la mediana (MAD) y clasificación celular por Kernel Ridge.
<code>ftirlib_0.0.4</code>	20 de junio de 2026	Versión final de producción. Desacoplamiento del módulo estadístico (<code>stats.py</code>), implementación de PCA mediante SVD, soporte nativo JCAMP-DX y la API variádica <code>plot_config</code> .

10.2. Minimización y Recreación de Experimentos Históricos

Con el fin de validar el comportamiento exacto de los algoritmos implementados a lo largo del historial del proyecto, se ha recreado una suite de experimentos sobre la muestra patrón de Óxido de Grafeno (`GO coque s lav ox.txt`). Cada experimento se ha minimizado mediante una galería gráfica en disposición 3×1 , contrastando la señal cruda o de entrada (izquierda), la superposición de señales con la estimación intermedia (centro) y el resultado espectral procesado y depurado (derecha).

10.2.1. Evaluación Comparativa de Algoritmos de Suavizado

El suavizado local atenúa el ruido instrumental de alta frecuencia. En esta prueba, se aplica sobre el espectro crudo en escala de Absorbancia, evaluando los tres algoritmos disponibles:

1. **Savitzky-Golay** (Ventana $W = 15$, Polinomio grado $d = 2$): Conserva la altura y la anchura de los picos originales sin introducir distorsiones por ensanchamiento de banda, gracias a su base de regresión polinómica local.

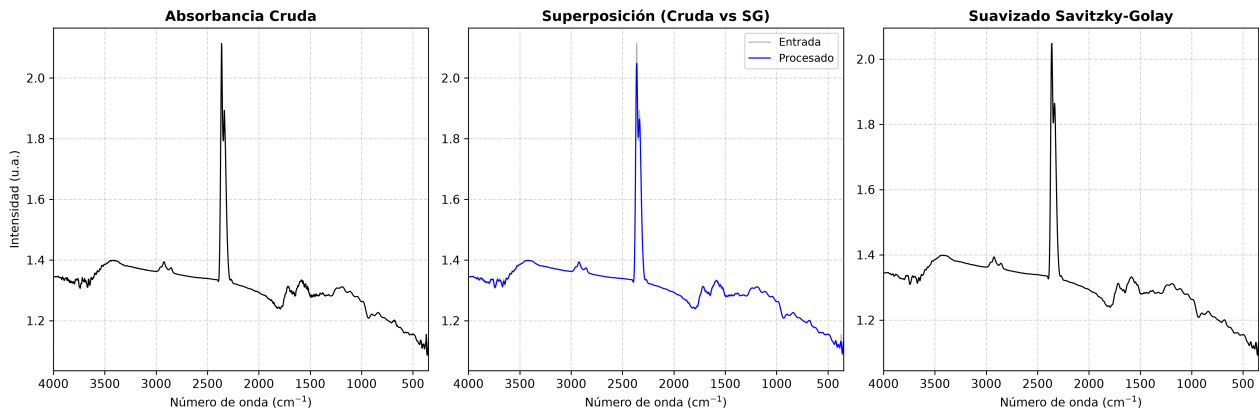


Figura 59: Recreación de suavizado mediante filtro Savitzky-Golay ($W = 15, d = 2$). Izquierda: Absorbancia cruda. Centro: Superposición de señal cruda y suavizada. Derecha: Espectro de absorbancia resultante.

2. **Promedio Móvil** (Ventana $W = 15$): Atenúa el ruido de forma uniforme promediando localmente los puntos de la ventana, lo que genera una ligera pérdida de definición en las cúspides de picos muy agudos.

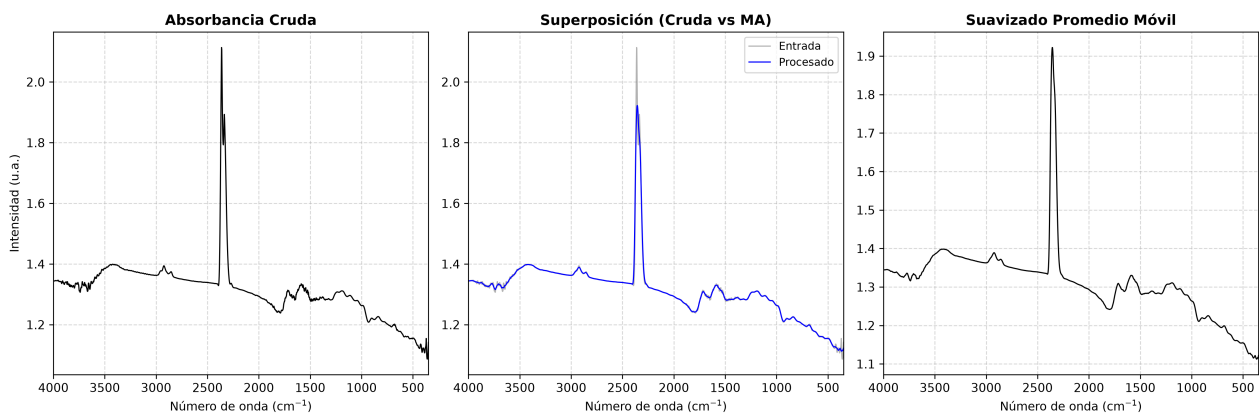


Figura 60: Recreación de suavizado por Promedio Móvil ($W = 15$). Izquierda: Absorbancia cruda. Centro: Superposición. Derecha: Espectro resultante con ligera atenuación de extremos.

3. **Filtro de Percentil** (Ventana $W = 15$, Percentil $P = 75$): Introduce un desplazamiento no lineal al seleccionar el percentil superior de la ventana local, útil para aislar y remover ruido asimétrico o de tipo impulso.

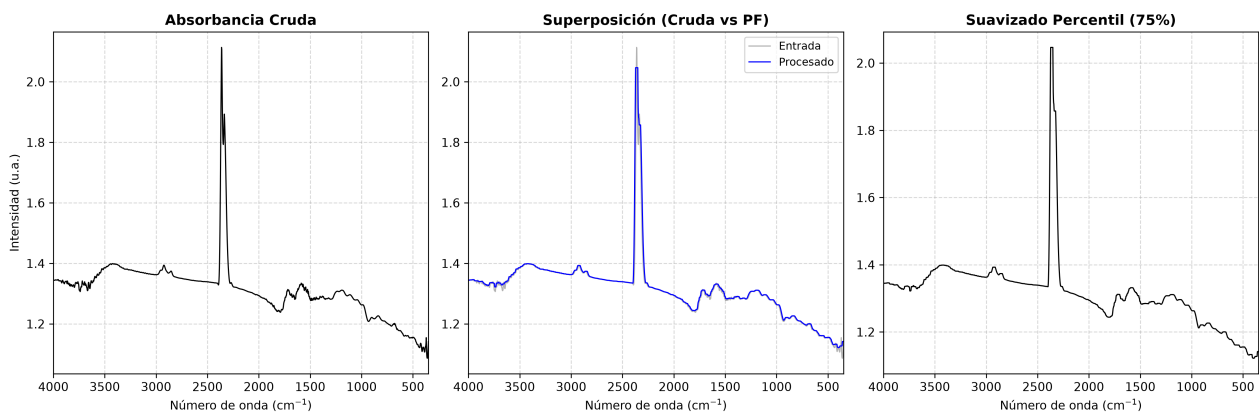


Figura 61: Recreación de suavizado por Filtro de Percentil ($W = 15, P = 75$). Izquierda: Absorbancia cruda. Centro: Superposición con desplazamiento no lineal. Derecha: Espectro resultante.

10.2.2. Evaluación Comparativa de Algoritmos de Corrección de Línea Base

La fluorescencia de fondo y la dispersión de luz provocan una deriva de la línea base en los espectros experimentales. A partir del espectro suavizado por el filtro Savitzky-Golay, se aplicaron los tres algoritmos de mínimos cuadrados penalizados asimétricos para remover el fondo (utilizando un parámetro común de penalización de rugosidad $\lambda = 10^5$):

1. **Mínimos Cuadrados Asimétricos (AsLS)** ($p = 0,001$, $\lambda = 10^5$): Ajusta una envolvente inferior suave aplicando pesos fijos asimétricos (p para puntos por encima del ajuste, y $1 - p$ para puntos por debajo).

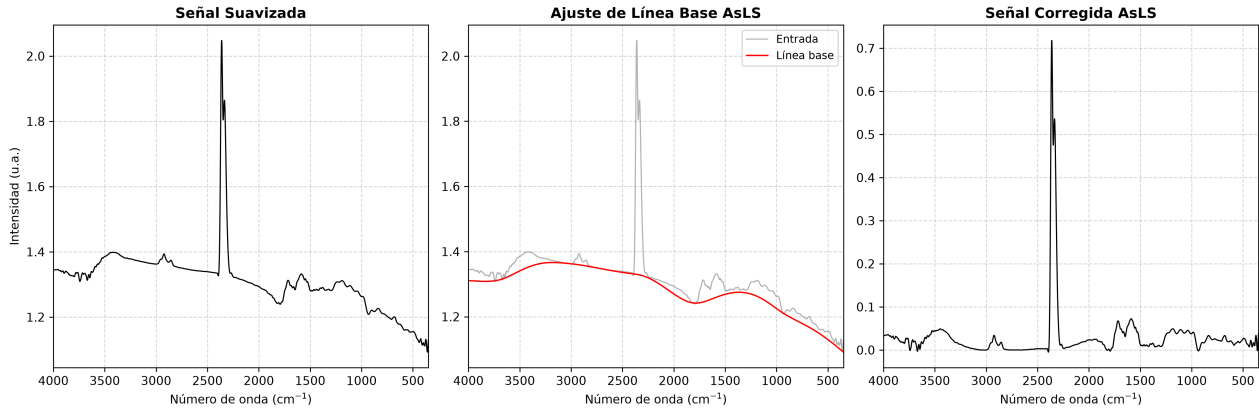


Figura 62: Recreación de corrección por AsLS ($\lambda = 10^5$, $p = 0,001$). Izquierda: Señal de absorbancia de entrada. Centro: Ajuste de la línea base (rojo) sobre la señal. Derecha: Señal neta resultante libre de deriva.

2. **Mínimos Cuadrados Penalizados Adaptativos (airPLS)** ($\lambda = 10^5$): Elimina la necesidad del parámetro manual p actualizando los pesos de forma iterativa y adaptativa mediante funciones exponenciales basadas en los residuos negativos.

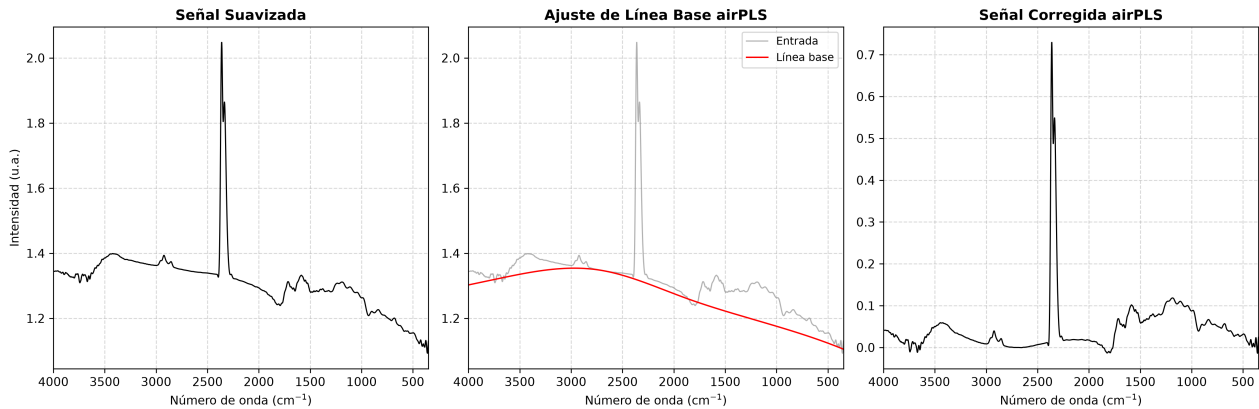


Figura 63: Recreación de corrección por airPLS ($\lambda = 10^5$). Izquierda: Señal de entrada. Centro: Línea base estimada adaptativamente (rojo). Derecha: Señal limpia resultante.

3. **Mínimos Cuadrados Penalizados Asimétricos Locales (arPLS)** ($\lambda = 10^5$): Utiliza una función de distribución logística continua para la actualización de pesos basada en la media y desviación estándar de los residuos negativos, logrando un ajuste suave y muy insensible a la presencia de picos analíticos anchos.

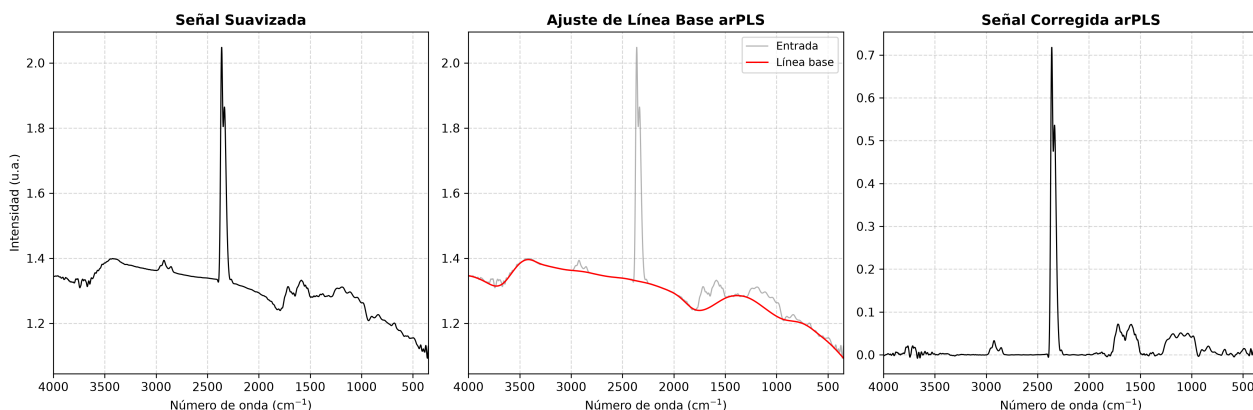


Figura 64: Recreación de corrección por arPLS ($\lambda = 10^5$). Izquierda: Señal de entrada. Centro: Línea base logística (rojo). Derecha: Espectro limpio libre de fluorescencia.

Como se aprecia en la galería experimental recreada, los algoritmos de preprocesamiento resuelven satisfactoriamente tanto la atenuación del ruido electrónico de alta frecuencia como la remoción matemática de la fluorescencia instrumental. La suite de pruebas unitarias automatizadas valida la reproducibilidad exacta de estos gráficos en cada compilación del sistema, consolidando las bases físicas y matemáticas de la biblioteca.

11. Conclusiones

La biblioteca `ftir_library` v0.0.4 consolida un entorno unificado y maduro para la caracterización, control de calidad y diagnóstico asistido por computadora utilizando espectroscopía vibracional (FTIR y Raman SERS).

El desacoplamiento de la arquitectura a través del módulo `stats.py` permite reutilizar de manera eficiente algoritmos avanzados como la proyección de componentes principales (PCA) mediante SVD y clasificadores regularizados multiclase (Kernel Ridge Classifier), habilitando flujos de trabajo científicos flexibles. El soporte nativo para el estándar industrial JCAMP-DX expande significativamente la utilidad de la biblioteca al permitir la ingesta directa de bases de datos de referencia clásicas como la del NIST.

Finalmente, la integración de una suite completa de pruebas automatizadas mediante `pytest` garantiza la estabilidad numérica y el correcto funcionamiento del pipeline frente a futuras extensiones, estableciendo una base sólida para el desarrollo de herramientas de diagnóstico biomédico y caracterización de nanomateriales en entornos de producción.

Referencias

- [1] Savitzky, A., & Golay, M. J. E. (1964). Smoothing and Differentiation of Data by Simplified Least Squares Procedures. *Analytical Chemistry*, 36(8), 1627-1639.
- [2] Eilers, P. H. C. (2003). A Perfect Smoother. *Analytical Chemistry*, 75(14), 3631-3636.
- [3] Zhang, Z.-M., Chen, S., & Liang, Y.-Z. (2010). An intelligent background correction method for vibrational spectroscopy. *Analyst*, 135(9), 2338-2346.
- [4] Erzina, M., Trelin, A., Guselnikova, O., Dvorankova, B., Strnadova, K., Perminova, A., Ulbrich, P., Mareš, D., Jerabek, V., Elashnikov, R., Svorcik, V., & Lyutakov, O. (2020). Precise cancer detection via the combination of functionalized SERS surfaces and convolutional neural network with independent inputs. *Sensors and Actuators B: Chemical*, 308, 127660.
- [5] Trelin, A. (2021). Cells Raman Spectra". *Kaggle Dataset*. URL: <https://www.kaggle.com/datasets/andriitrelin/cells-raman-spectra>
- [6] Sardinha, A. F., Almeida, D. A. L., & Ferreira, N. G. (2020). Electrochemical impedance spectroscopy correlation among graphene oxide/carbon fibers (GO/CF) composites and GO structural parameters produced at different oxidation degrees. *Journal of Materials Research and Technology*, 9(5), 10841-10853.
- [7] Sardinha, A. F. (2020). "FTIR spectra of Graphene oxide". *Mendeley Data*, V1. DOI: 10.17632/4j2hxyrcf6.1
- [8] Figueroa-Rosales, E. X., Hernández-Cruz, D., Robles-Águila, M. J., Rodríguez-García, M. E., & Martínez-Juárez, J. (2023). Stability of the Photoluminescent Response on Hydroxyapatite/Multi-Walled Carbon Nanotube Composites. *Photonics*, 10(8), 919.
- [9] Figueroa-Rosales, E. X. (2023). "FTIR & PL raw data". *Mendeley Data*, V1. DOI: 10.17632/mhk9g942sr.1
- [10] Whitaker, D. A., & Hayes, K. (2018). A simple algorithm for cosmic ray spike removal in Raman spectra. *Chemometrics and Intelligent Laboratory Systems*, 179, 82-84.
- [11] Baek, S.-J., Park, A., Shen, Y.-J., & Lim, J. (2015). A simple and effective baseline correction method for Raman spectra. *Analyst*, 140(1), 250-257.
- [12] Hoerl, A. E., & Kennard, R. W. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1), 55-67.